

Conception de circuits numériques complexes sur FPGA SN422

III Synthèse RTL

Louis Morge-Rollet

Bureau : A203

louis.morge-rollet@esisar.grenoble-inp.fr

(Ecrit par Vincent Beroulle et Louis Morge-Rollet)

Plan global du cours

I Introduction

II Conception de systèmes synchrones

III Synthèse RTL

Méthode « régulière »

Méthode « aléatoire » (ou à base de FSM)

Méthode « combinée » (ou à base de FSMD)

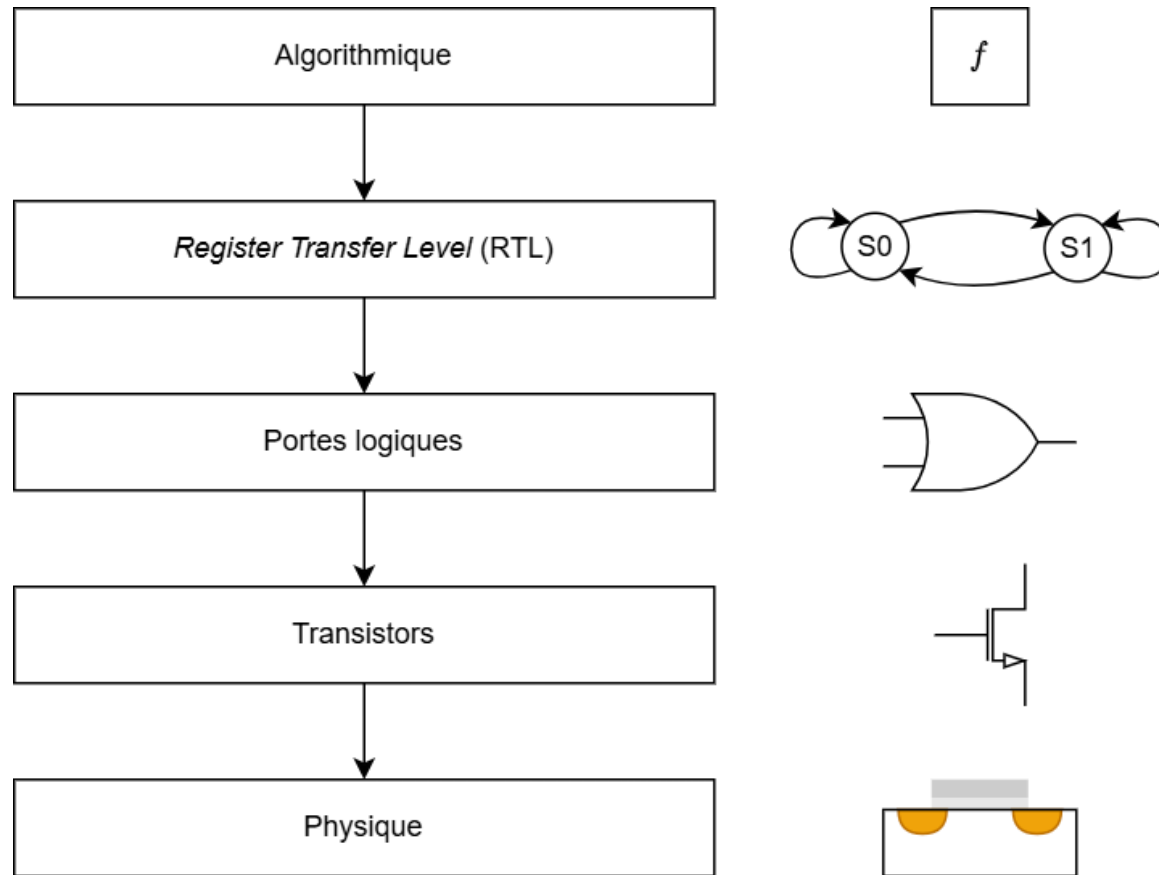
IV Synthèse de haut niveau (HLS)

V Conception de filtres

VI Flot de conception avancée

Introduction

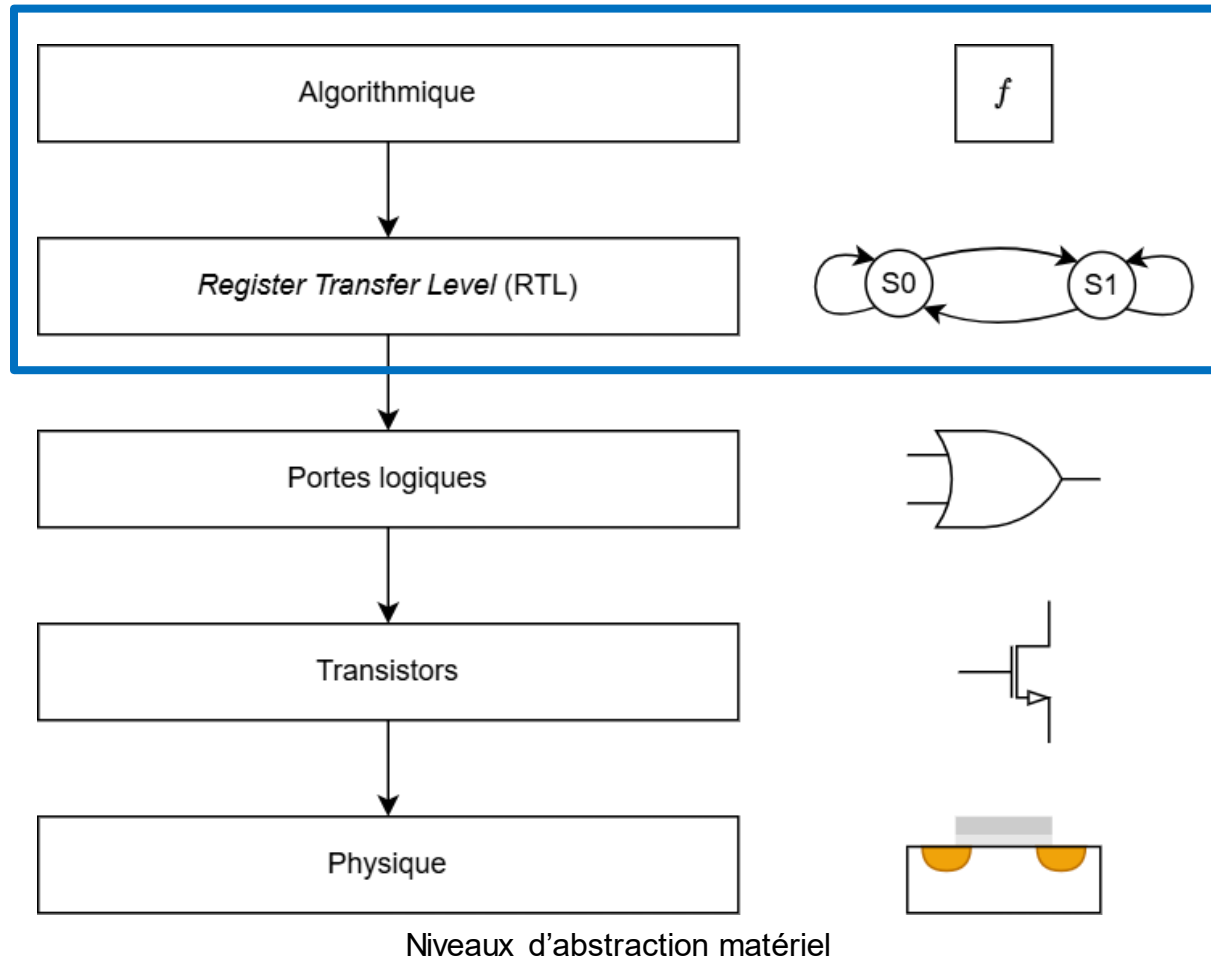
Rappel: Niveau d'abstraction



Niveaux d'abstraction matériel

Introduction

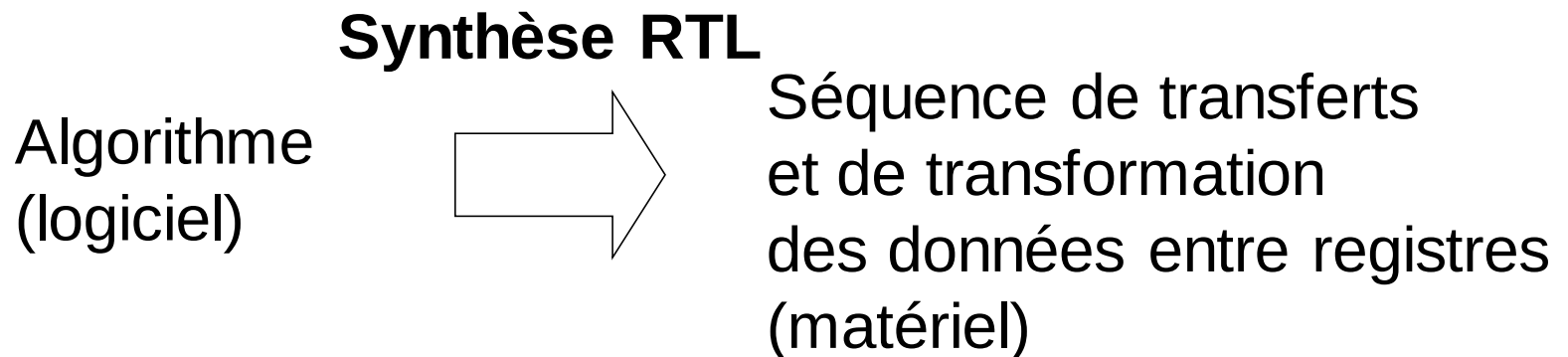
Rappel: Niveau d'abstraction



Introduction

Rappel: synthèse RTL

- Transformations qui, à partir d'une *spécification* algorithmique ***génère une description au niveau RTL*** (dans laquelle l'ordonnancement des opérations est clairement spécifié)



Introduction

Objectif : maîtriser la complexité

- Les circuits séquentiels peuvent être conçus avec trois méthodes différentes :
 - « *Approche régulière* » (à base de compteurs, de registres à décalage...)
 - l'état suivant est une « expression régulière » de l'état présent
 - « *Approche dite aléatoires* » (à base de FSM)
 - pas de relation spéciale entre le code binaire de l'état présent et celui de l'état futur
 - « *Approche Combinée* » (à base d'une FSMD : FSM avec *Data path*) : une combinaison des deux types précédents

L'approche combinée permet de concevoir des circuits séquentiels composés à la fois de machines à états et de composants réguliers

Introduction

Objectif : maîtriser la complexité

- « *Approche régulière* »
 - Approche intuitive qui ne nécessite pas de connaissance spécifique en conception
 - Mais approche qui nécessite une bonne maîtrise du VHDL et des outils de synthèse logique
- « *Approche dite aléatoires* »
 - Approche systématique basée sur des représentations graphiques
 - Mais approche limité aux circuits simples possédant peu d'états
- « *Approche Combinée* » (à base d'une FSMD : FSM avec Datapath)
 - Approche systématique basée sur des représentations graphiques
 - Approche qui peut s'appliquer aux circuits complexes

Cahier des charges

- Réaliser l'implémentation d'un générateur d'impulsion qui génère une impulsion de 5 périodes de large de l'horloge *clk*
- Spécifications détaillées :
 - 2 entrées *go* et *stop*, une sortie *pulse*
 - *go* est un signal de déclenchement (actif seulement quand l'impulsion est basse)
 - Si *stop* est actif alors le signal de sortie retourne à 0

Plan global du cours

- I Introduction
- II Conception de systèmes synchrones
- **III Synthèse RTL**
 - Méthode « régulière »
 - Méthode « aléatoire » (ou à base de FSM)
 - Méthode « combinée » (ou à base de FSMD)
- IV Synthèse de haut niveau
- V Conception de filtres

Composants réguliers

➤ Exemples:

- Registres: Une collection de **bascules D en parallèle**, qui sont pilotés par le même signal d'horloge, ainsi que le même signal de *reset*
- Registres à décalage: Une collection de **bascules D en série**, qui sont pilotés par le même signal d'horloge, ainsi que le même signal de *reset*
- Compteurs: Un composant synchrone permettant de **générer une séquence de nombres successifs** (et de recommencer)
- ...

Solution « régulière »

➤ Implantation régulière =

compteur modulo 5

+

circuit de contrôle spécifique
pour autoriser ou non le comptage

+

mémorisation de l'état
dans une bascule supplémentaire

Solution « régulière »

Code VHDL

```
process(clk)
```

```
  begin
```

```
    if clk'event and clk = '1' then
```

```
      if stop = '1' or count = 5 then
```

```
        pulse <= '0';
```

```
        front <= '0';
```

```
        count <= 0;
```

```
      elsif (front = '1' or go = '1') then
```

```
        pulse <= '1';
```

```
        front <= '1';
```

```
        count <= count + 1;
```

```
      end if;
```

```
end process;
```

Remise à zéro

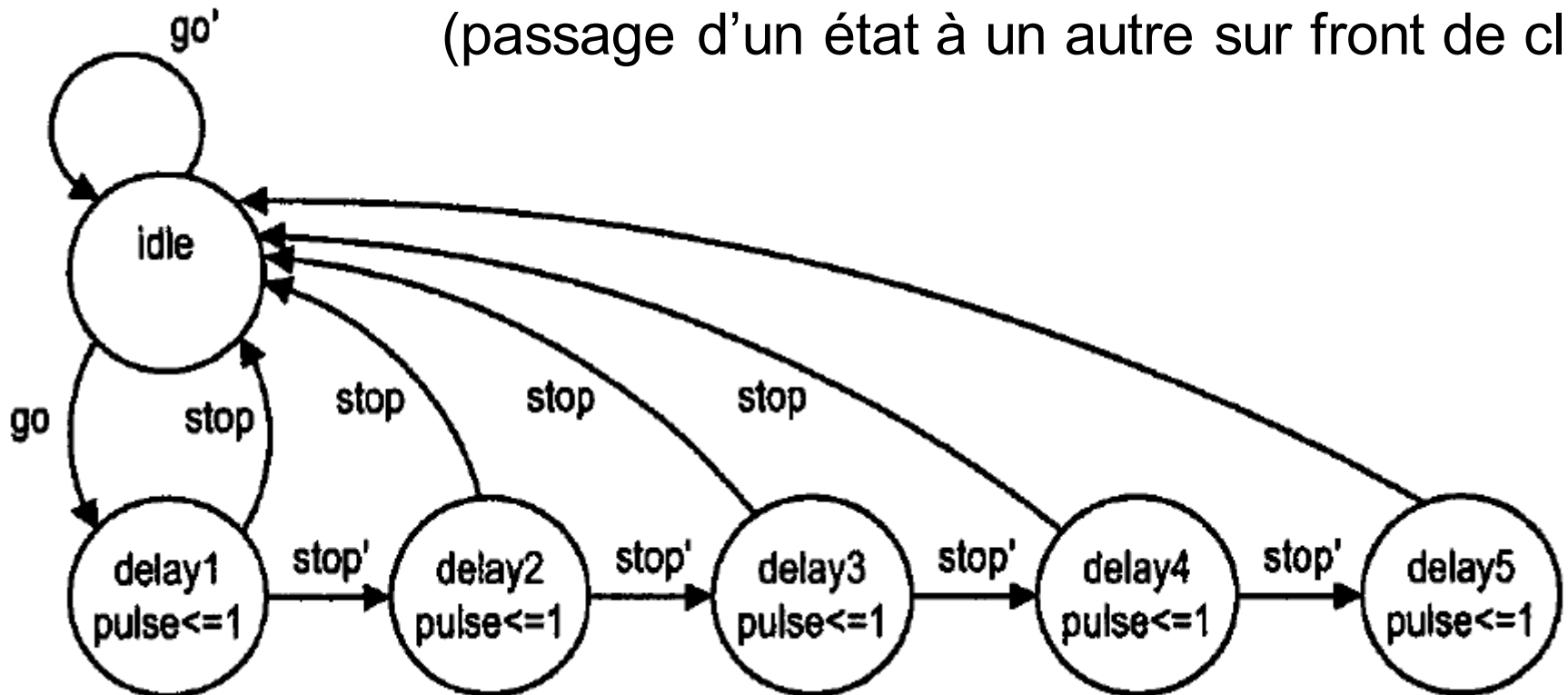
Génération de
l'impulsion et
comptage

Plan global du cours

- I Introduction
- II Conception de systèmes synchrones
- **III Synthèse RTL**
 - Méthode « régulière »
 - **Méthode « aléatoire » (ou à base de FSM)**
 - Méthode « combinée » (ou à base de FSMD)
- IV Conception de filtres

Solution « aléatoire » (ou FSM)

Par défaut: $\text{pulse} \leq 0$
(passage d'un état à un autre sur front de clk)



Description de FSM

Objectifs

- Les machines à états (FSM) sont des composants couramment utilisés, nous allons voir comment :
 - Les décrire pour atteindre des performances optimales (notion de compromis Temps/Fréquence)

Description de FSM

Rappels

- On utilise un graphe d'états ou diagramme d'états(-transitions) :
 - État : 1 rond + 1 nom à l'intérieur (le nom de l'état)
 - État initial : « flèche venant de nulle part avec rst » et/ou double cercle
 - Transition : une flèche portant la valeur condition associée à cette transition, la transition n'est franchie que lors du front actif de l'horloge
 - Sortie :
 - soit associée à l'état (Moore)
 - soit associée à une transition (e/s) (Mealy)

➤ Exemple :

Quand il n'y a pas de condition alors la condition est toujours vraie

La sortie ne dépend que de l'état

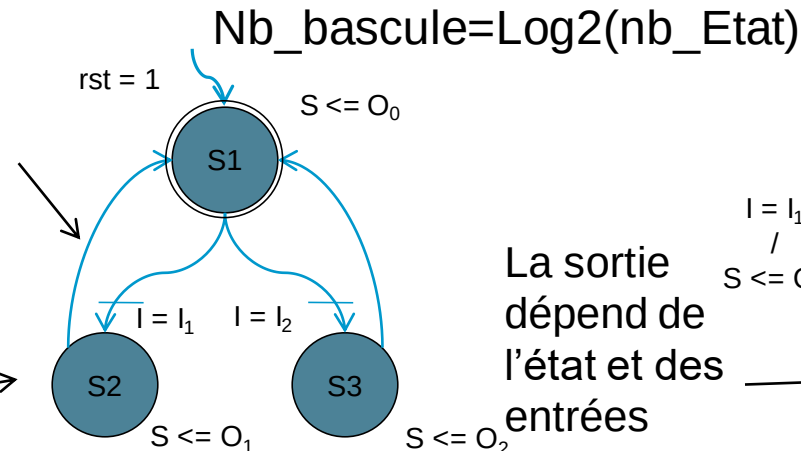


Diagramme d'états d'une machine de Moore :
1 état = 1 sortie

Le reset peut être synchrone ou asynchrone (il faut l'indiquer explicitement)

La valeur de la sortie si $I=I2$

La sortie dépend de l'état et des entrées

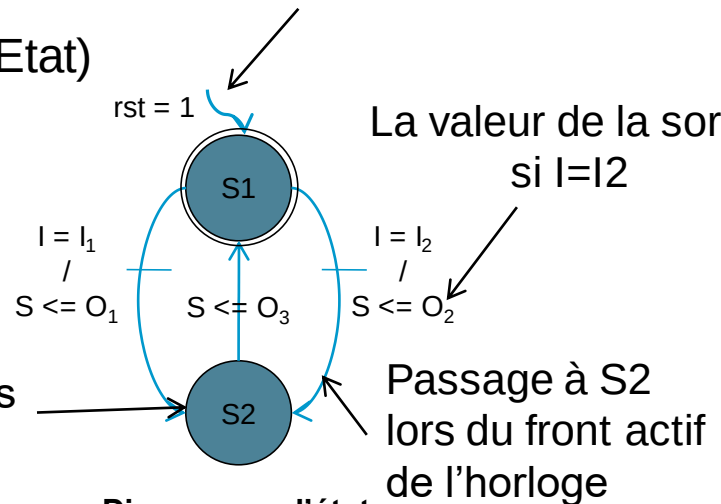
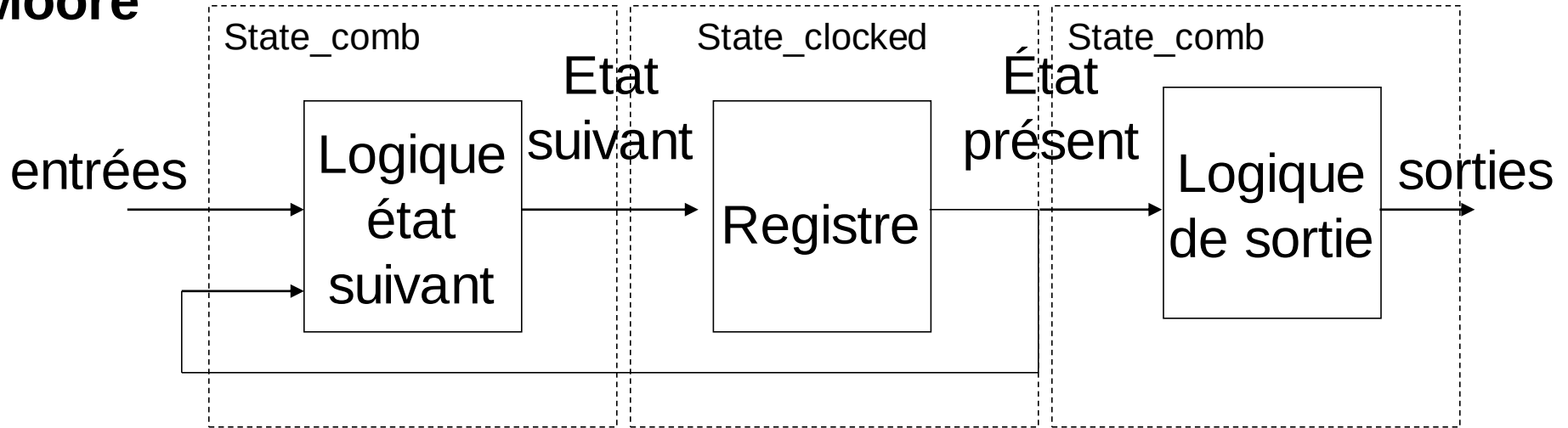


Diagramme d'états d'une machine de Mealy :
1 transition = 1 sortie

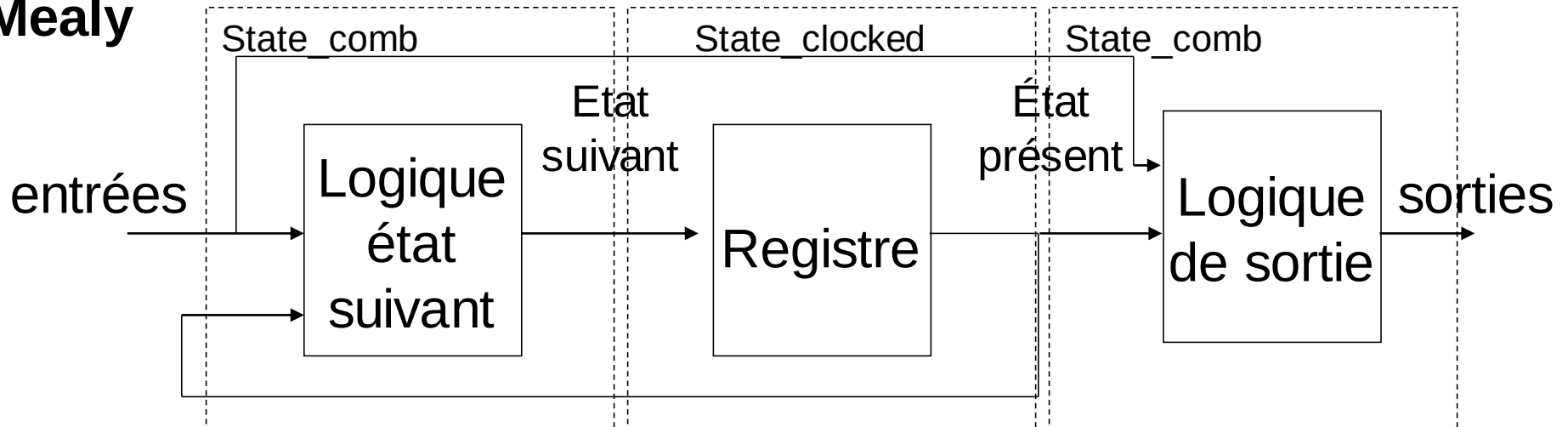
Description de FSM

Description de FSM à deux processus

Moore



Mealy



Description de FSM

FSM à deux process (une description possible)

-- déclaration des ports d'entrées/sorties de la FSM

entity generateur **is port** (

go, stop, clk : **in bit**;

pulse : **out bit**);

end generateur;

architecture state_machine **of** generateur **is**

-- Déclaration d'un type énuméré composé des noms des états

type StateType **is** (idle, delay1, delay2, delay3, delay4, delay5);

-- Déclaration de deux signaux du type précédent

signal present_state, next_state : StateType;

begin

Description de FSM

FSM à deux process

```
state_comb:process(present_state, go, stop)
begin
    next_state <= idle;
    case present_state is
        when idle => pulse <= '0';
            if go = '1' then
                next_state <= delay1;
            --else
            --    next_state <= idle;
            end if;
        when delay1 => pulse <= '1';
            if stop = '1' then
                next_state <= idle ;
            else
                next_state <= delay2;
            end if; ...
    end case;
end process;
```

Dans chaque état
on définit
toutes les sorties
et les états
suivants
accessibles en
fonction des entrées

Description de FSM

FSM à deux process

```
state_clocked:process(clk) begin
    if (clk'event and clk='1') then
        present_state <= next_state;
    end if;
end process state_clocked;
```

Second *process* qui décrit le passage d'un état à l'autre sur les fronts montants de l'horloge

```
end architecture state_machine;
```

Remarque :

- **Cette description correspond à une machine de *Moore*** car les sorties dépendent uniquement de l'état de la machine et pas des entrées.
- Pour une machine de *Mealy*, on aurait des *if...then* ou des *case...* pour affecter les sorties en fonction des entrées dans le premier *process*

Description de FSM

FSM à deux *process*

- Problème de la mémorisation implicite :
 - Dans un *process* « non clocké » (combinatoire), si l'affectation d'un signal est décrite seulement sous certaines conditions, alors **la synthèse produit des *latches* (basculs asynchrones, non désirées!) pour mémoriser l'état du signal dans tous les cas non spécifiés**
- Dans le *process state_comb* (« non clocké »), **il faut affecter toutes les sorties dans toutes les conditions** ou mettre des valeurs par défauts en début de processus

Description de FSM

FSM à deux process

➤ Problème du reset :

- La simulation utilise comme valeur initiale les valeurs à gauche des énumérés : ici, *idle* est l'état initial de la machine
- Après synthèse, selon la technologie (ASIC, ...), l'état initial peut être n'importe quel état (et dans certains cas, cet état peut ne pas être atteignable par la machine), aussi il est recommandé de **toujours ajouter un reset dans le processus synchrone**

Description de FSM

FSM à deux process – Reset Asynchrone

```
process (clk, rst)
begin
  if (rst = '1') then
    present_state <= idle;
  elsif (clk'event and clk='1') then
    present_state <= next_state;
  end if;
end process;
```

Description de FSM

FSM à deux process – Reset Synchrone

```
process (clk)
begin
if (clk'event and clk='1') then
    if (rst = '1') then
        present_state <= idle;
    else
        present_state <= next_state;
    end if;
end if;
end process;
```

Description de FSM

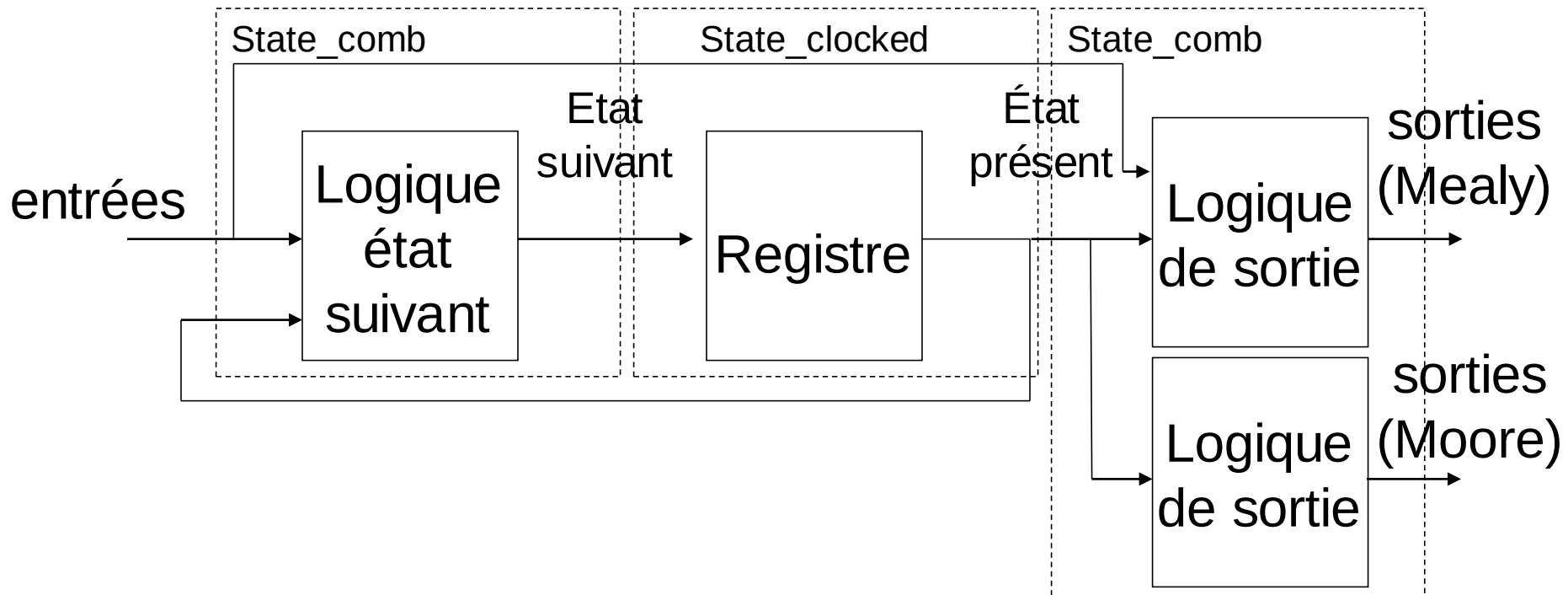
Directive de synthèse – Choix du codage des FSM

- **Par exemple, codage « *One-Hot* » des états :**
 - N bascules représentent une FSM à n états
 - Chaque état a sa propre bascule
 - Une seule bascule est à '1' à la fois (elle est dite « chaude »)
- L'avantage de ce code réside **généralement** dans le fait que **les logiques de l'état suivant et de la sortie sont plus simples** (vitesse accrue au prix d'un grand nombre de bascules...) :
 - **Plus adapté aux FPGA**
 - Seulement une directive lors de la synthèse

Description de FSM

Généralisation

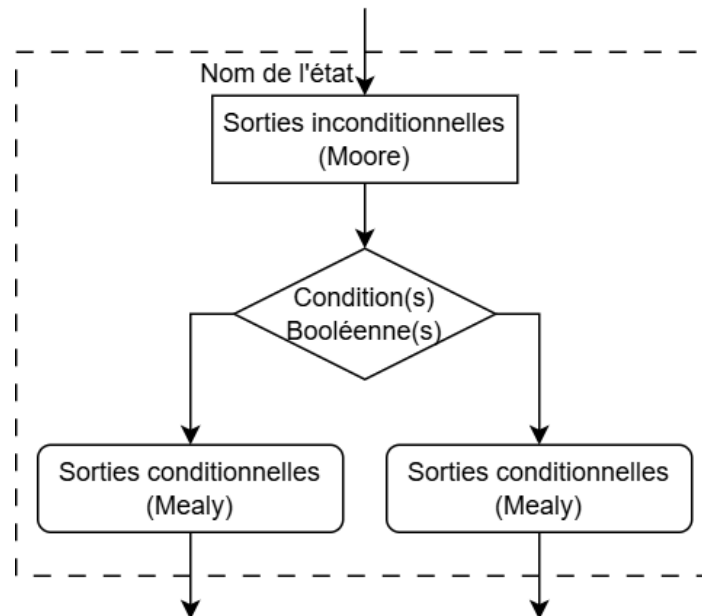
- En réalité, les FSM peuvent avoir des sorties:
 - Inconditionnelles (Moore), dépendantes de l'état présent
 - Conditionnelles (Mealy), dépendantes de l'état présent et des entrées



Description de FSM

Algorithm State Machine (ASM)

- Une alternative au diagramme d'états est le digramme *Algorithm State Machine (ASM)*
 - L'ASM est plus adapté à l'écriture du VHDL, ainsi que pour représenter des transitions complexes



Représentation d'un état sous forme d'un bloc ASM

Description de FSM

Algorithm State Machine (ASM)

- Le diagramme ASM est composé de blocs ASM (*ASM Blocks*, **carré en pointillé**), eux-mêmes composé de trois types de sous-blocs:
 - Sous-bloc d'état (*state box*, **carré en trait plein**, obligatoire), contenant les sorties de Moore.
 - Sous-bloc de décision (*decision box*, **losange en trait plein**, optionnel), dépendant des entrées.
 - Sous-bloc conditionnel (*conditional box*, **carré en trait plein avec des bords arrondis**, optionnel), contenant les sorties de Mealy.
- Les transitions entre blocs sont synchrones.

Description de FSM

Algorithm State Machine (ASM)

- Règle n°1: Une transition entre deux blocs ASM arrive forcément dans un sous-bloc d'état.

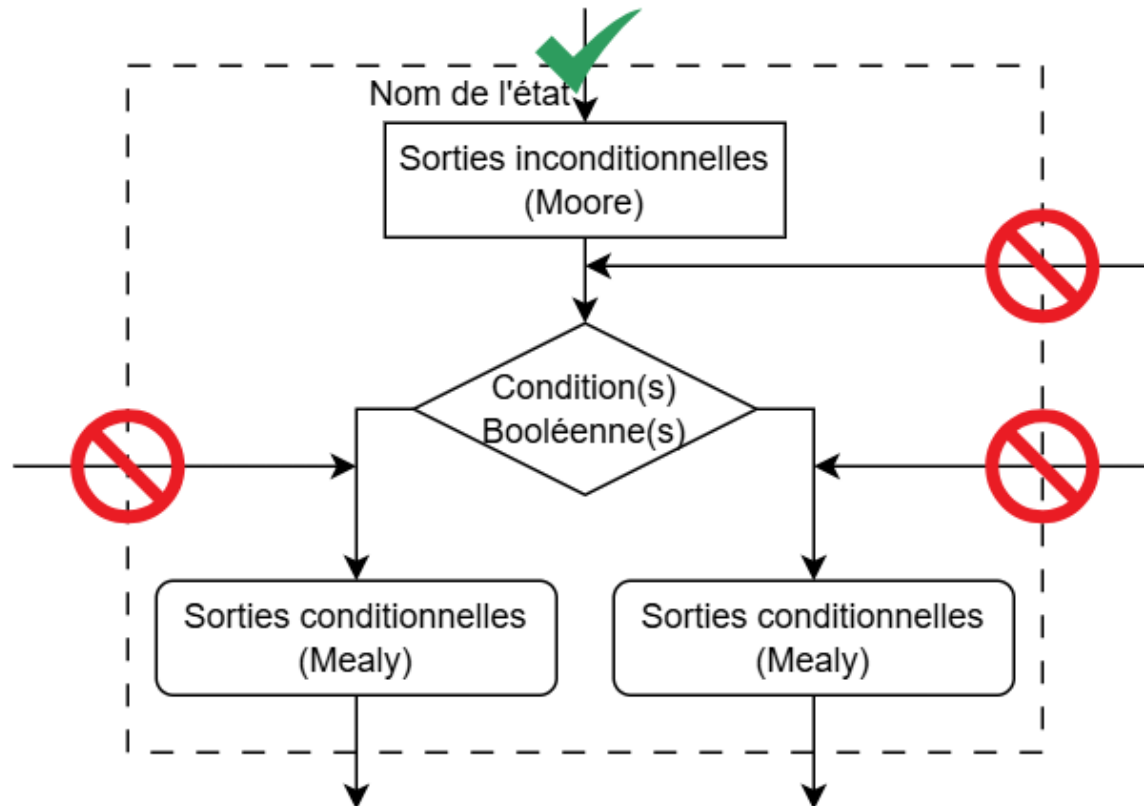


Illustration de la règle n°1 des ASM

Description de FSM

Algorithm State Machine (ASM)

- Règle n°2: Une sortie est soit inconditionnelle (Moore), soit conditionnelle (Mealy).

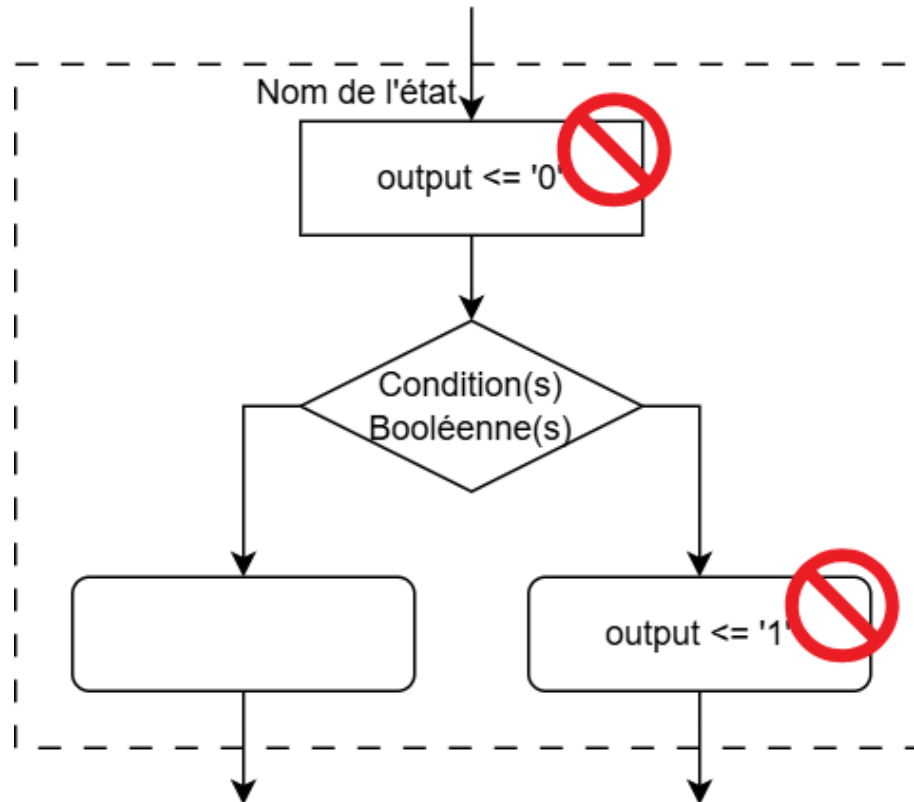


Illustration de la règle n°2 des ASM

Description de FSM

Algorithm State Machine (ASM)

- Règle n°2: Une sortie est soit inconditionnelle (Moore), soit conditionnelle (Mealy).

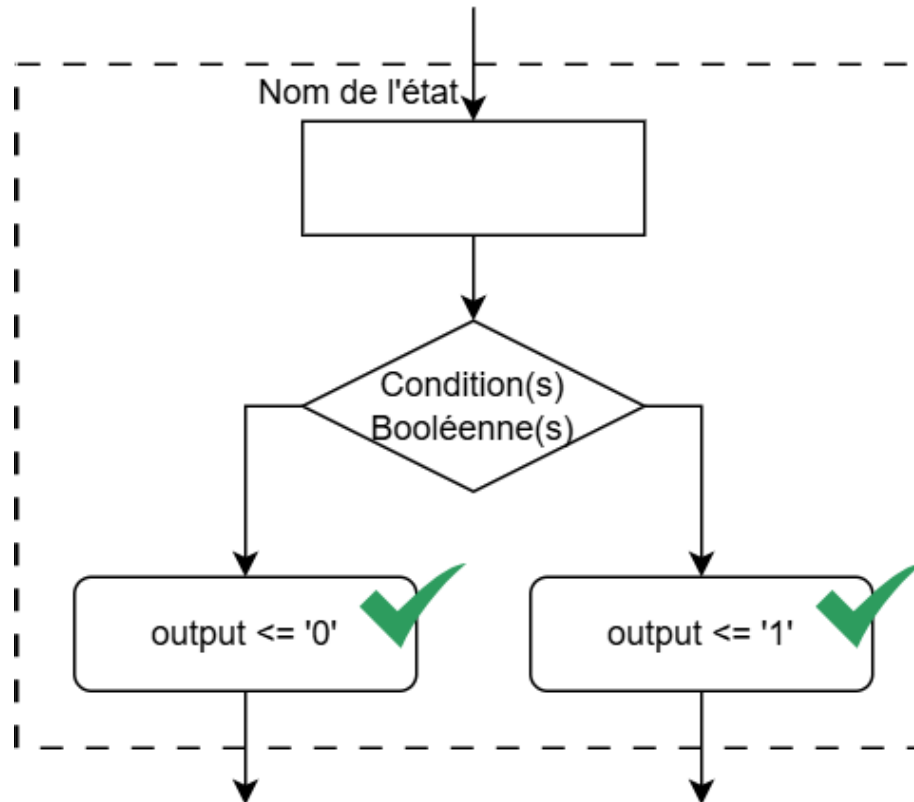


Illustration de la règle n°2 des ASM

Description de FSM

Algorithm State Machine (ASM)

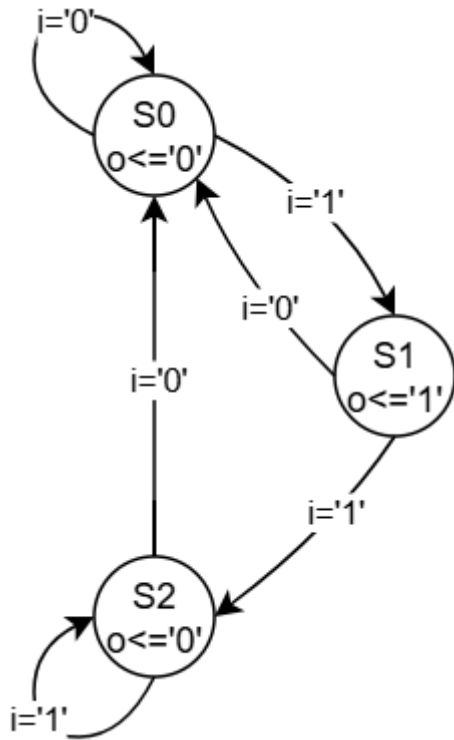
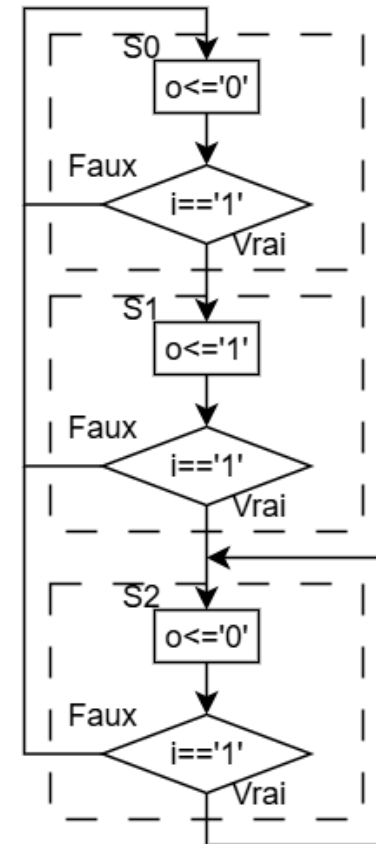


Diagramme d'état d'un détecteur de front montant à sortie de Moore



ASM d'un détecteur de front montant à sortie de Moore

Description de FSM

Algorithm State Machine (ASM)

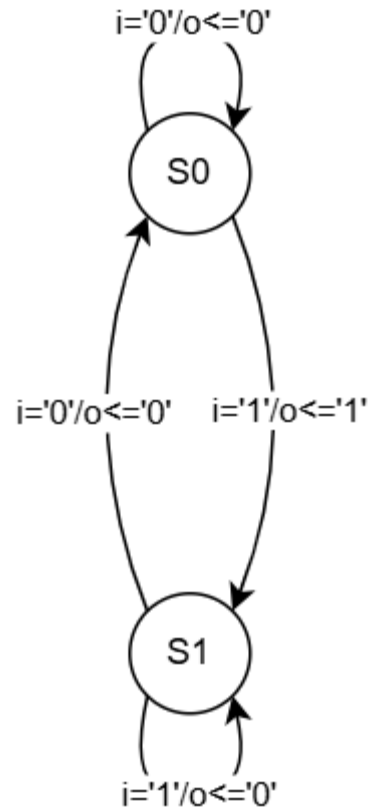
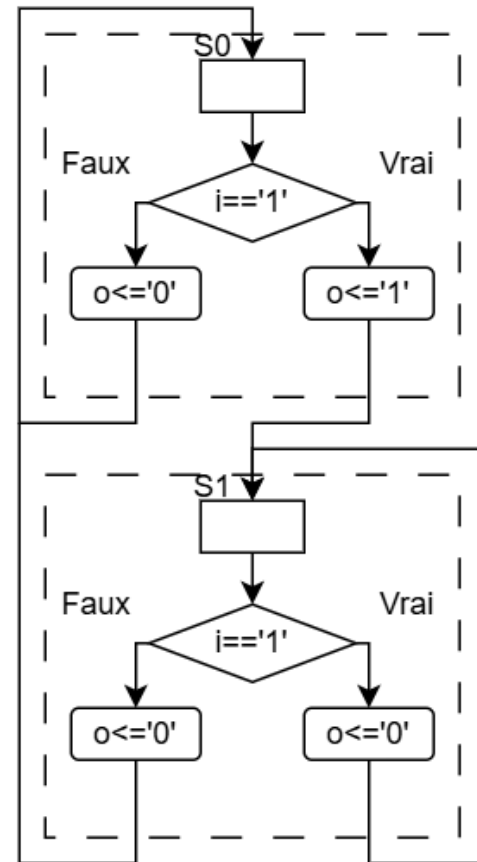
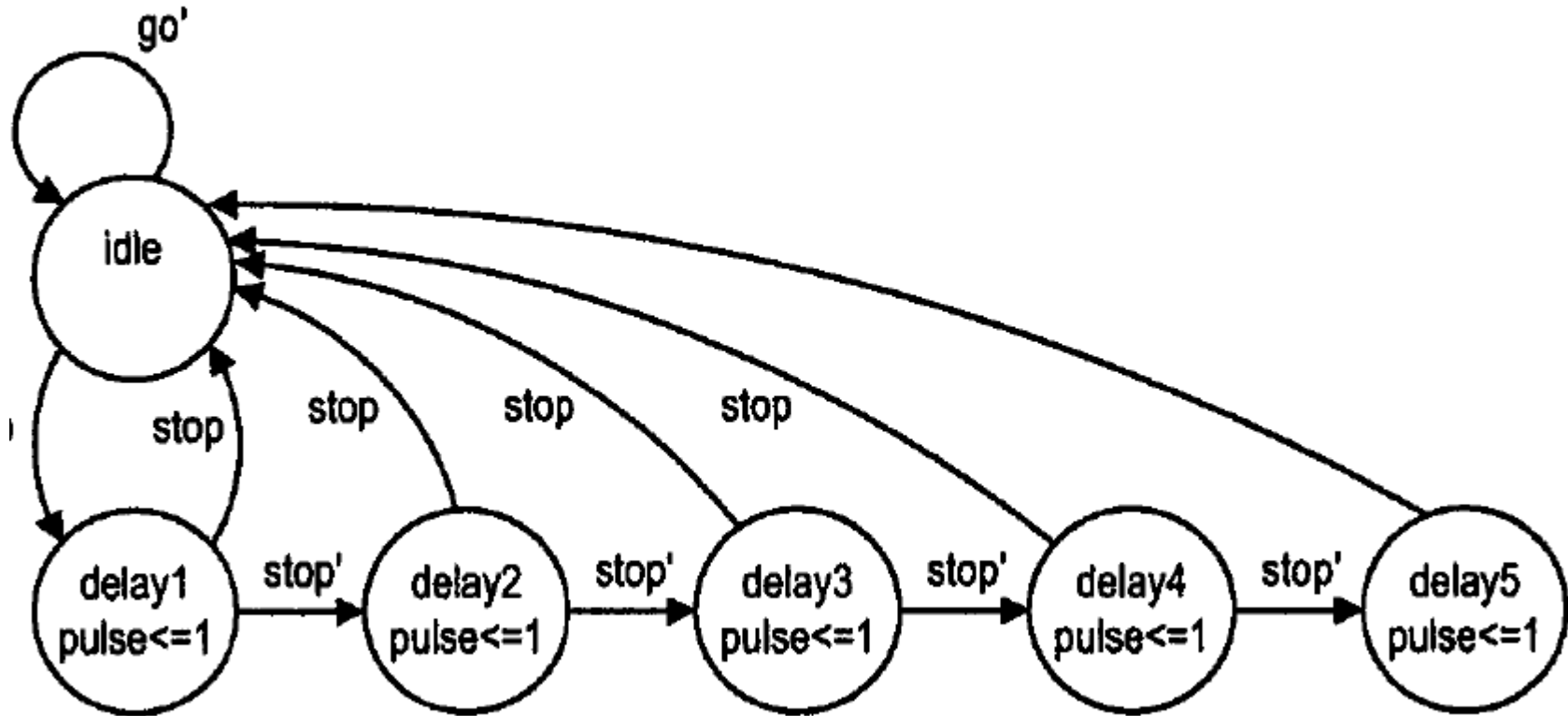


Diagramme d'état d'un détecteur de front montant à sortie de Mealy



ASM d'un détecteur de front montant à sortie de Mealy

Solution « aléatoire » (ou FSM)



**Mais que se passe t-il s'il y a trop d'états?
(par exemple, pour un générateur d'impulsions de
1000 périodes de large)**

Plan global du cours

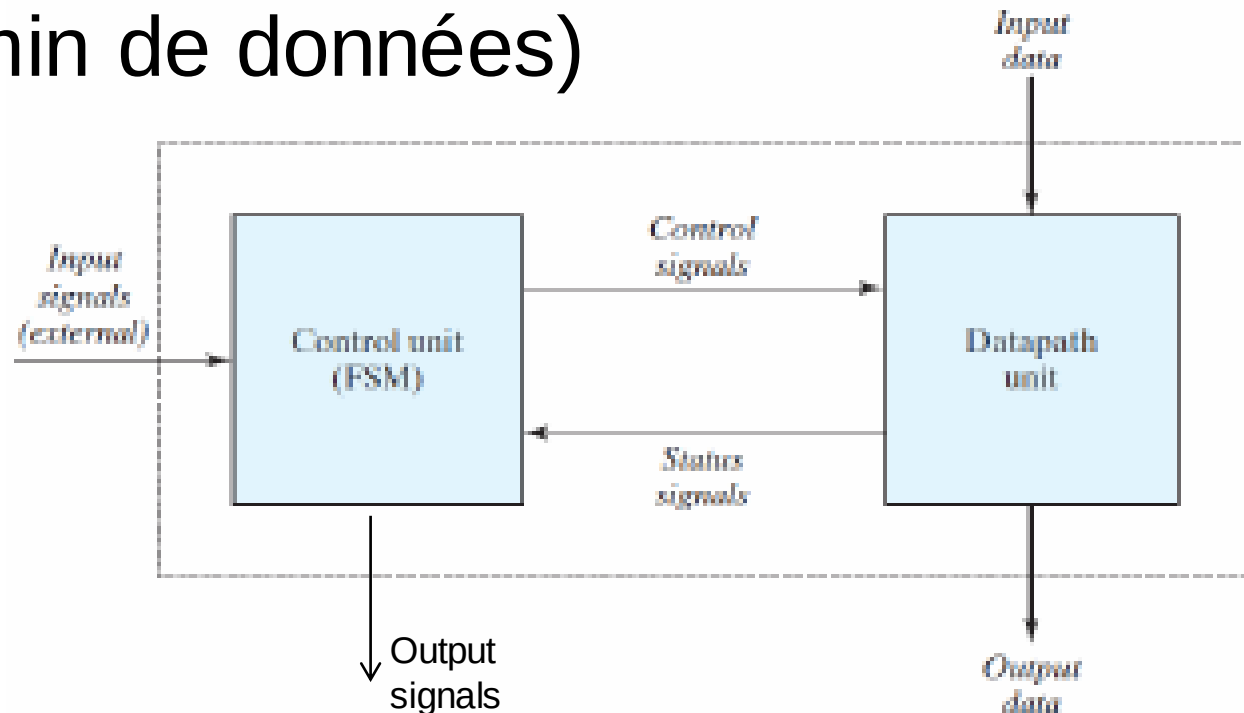
- I Introduction
- II Conception de systèmes synchrones
- **III Synthèse RTL**
 - Méthode « régulière »
 - Méthode « aléatoire » (ou à base de FSM)
 - **Méthode « combinée » (ou à base de FSMD)**
- IV Conception de filtres

Méthode combinée (ou FSMD)

Introduction

- Méthode de conception RTL qui permet de concevoir un circuit séquentiel en combinant à la fois machine à états (FSM) et composants réguliers (chemin de données)

➔ Réduction du nombre d'états + régularité des composants



Méthode combinée (ou FSMD)

Introduction

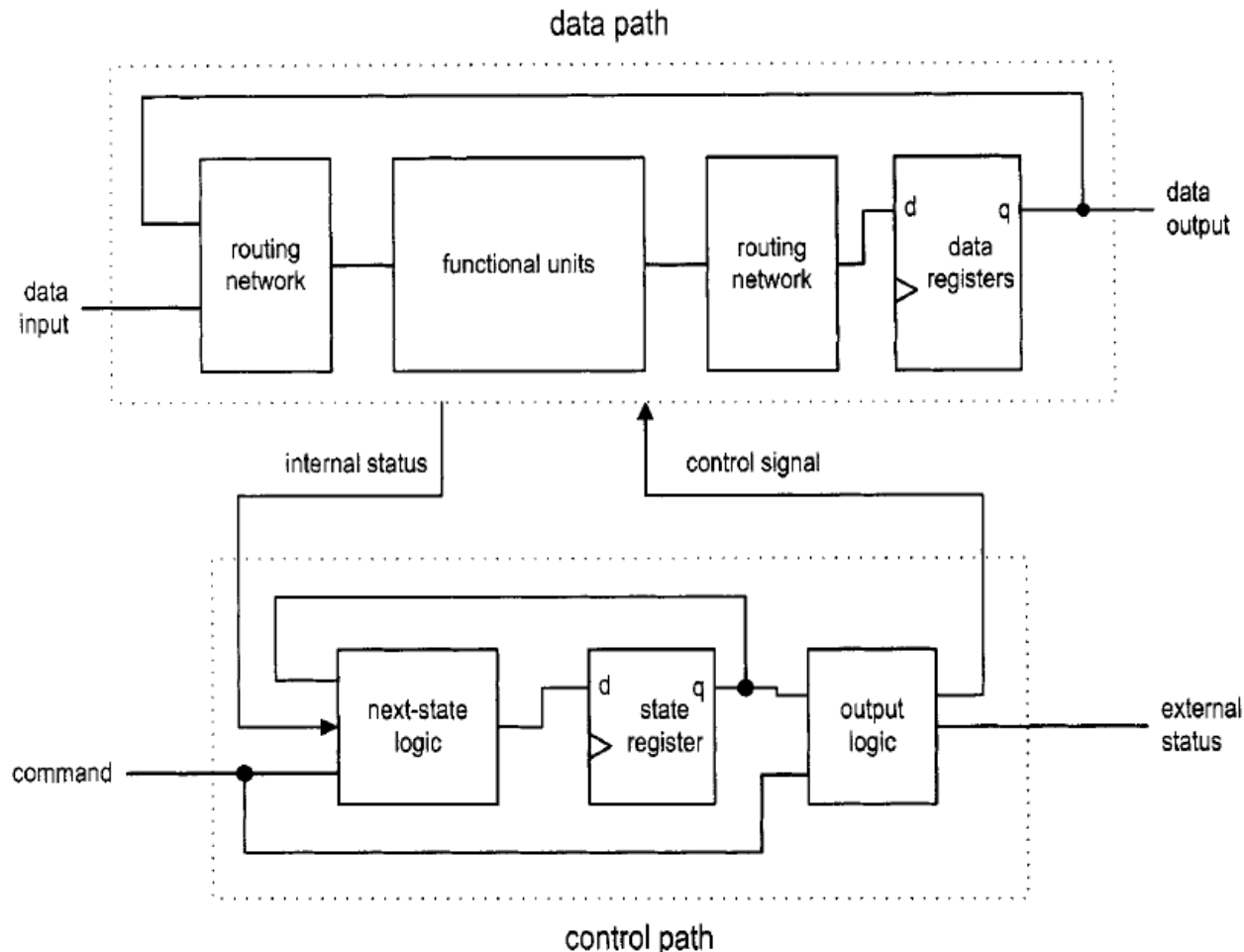
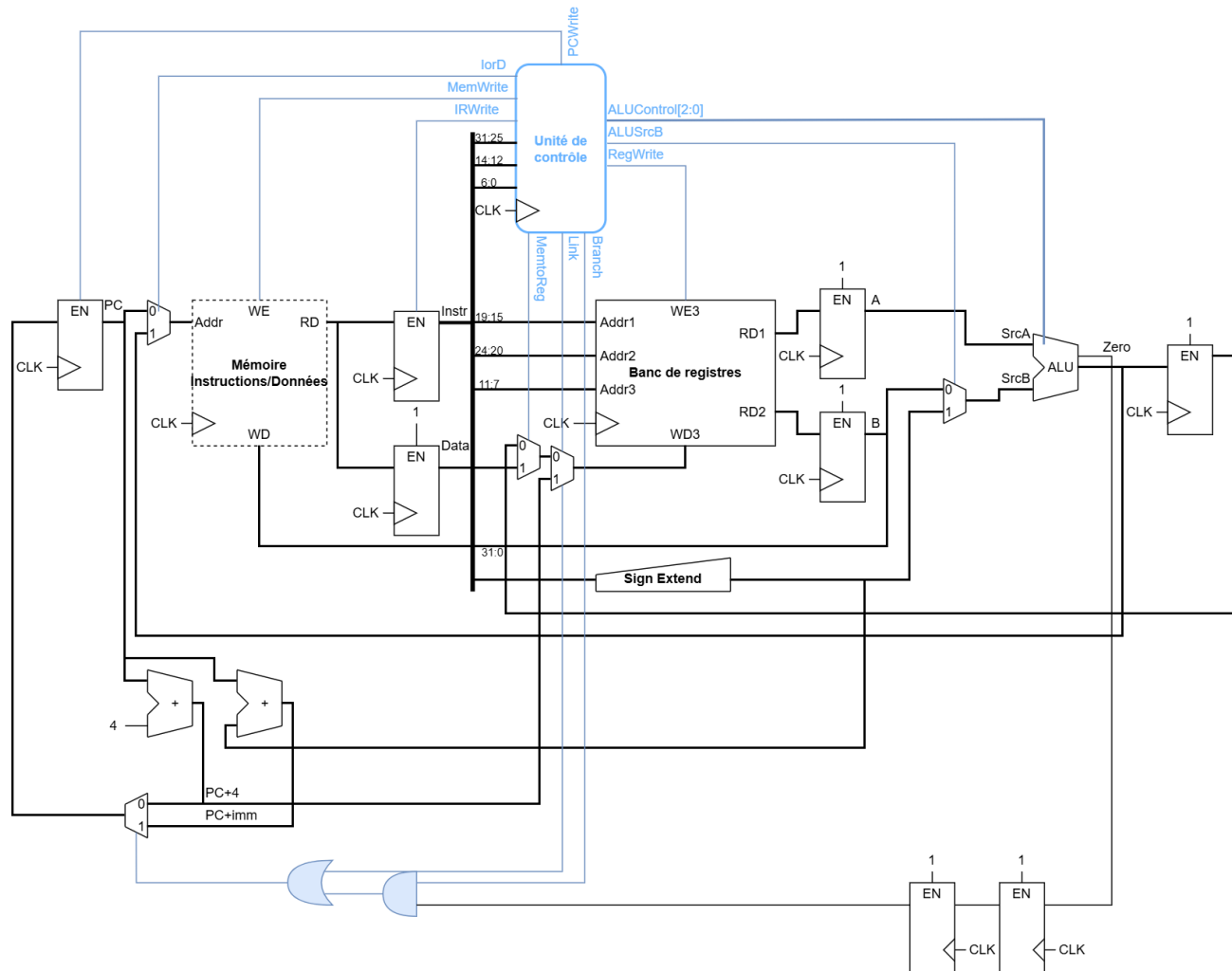


Illustration du fonctionnement interne d'une FSMD ([Pong P. Chu](#))

Méthode combinée (ou FSMD)

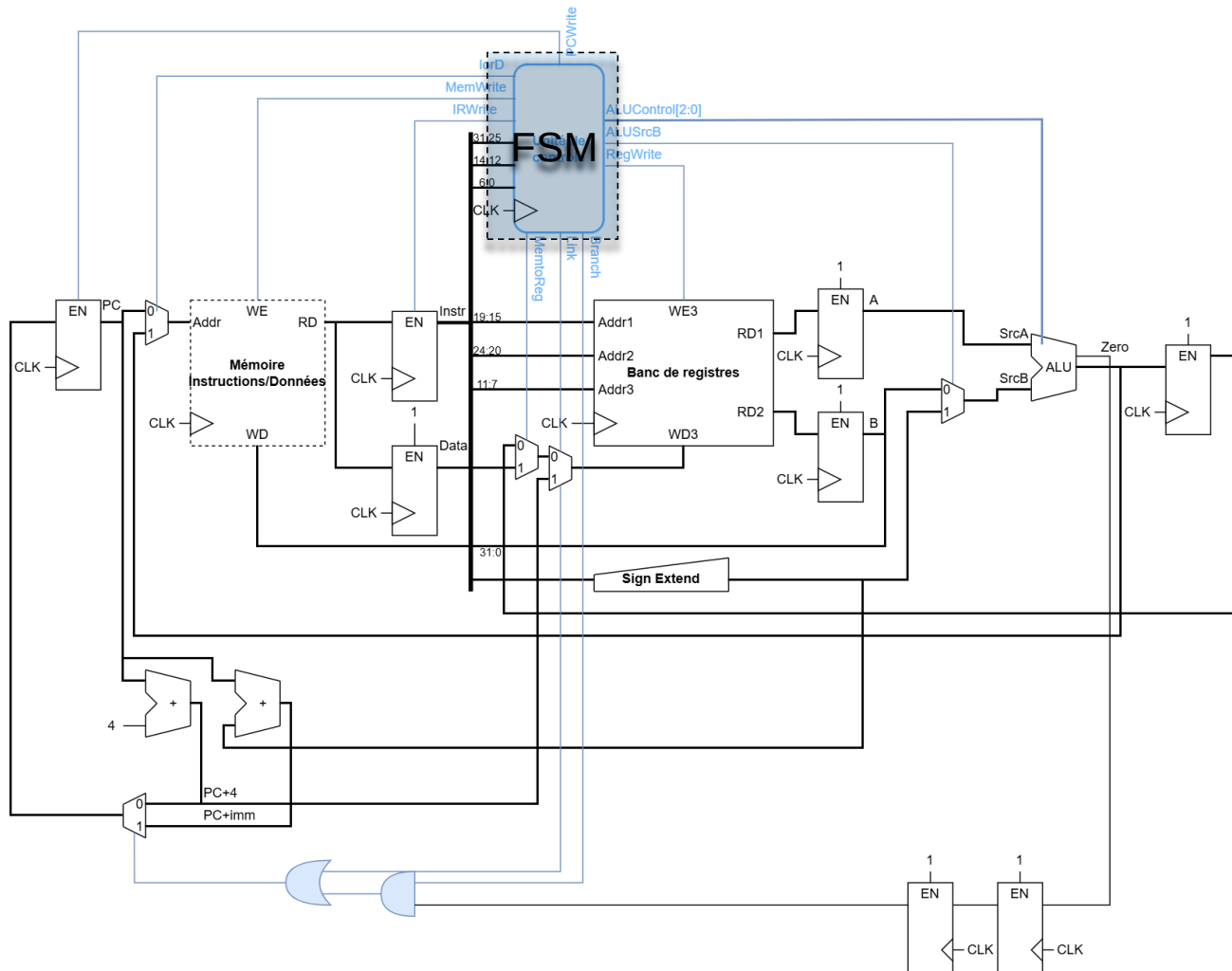
Exemple



Exemple d'une FSMD pour un processeur RISC-V multi-cycles (avec mémoire, PSN10)

Méthode combinée (ou FSMD)

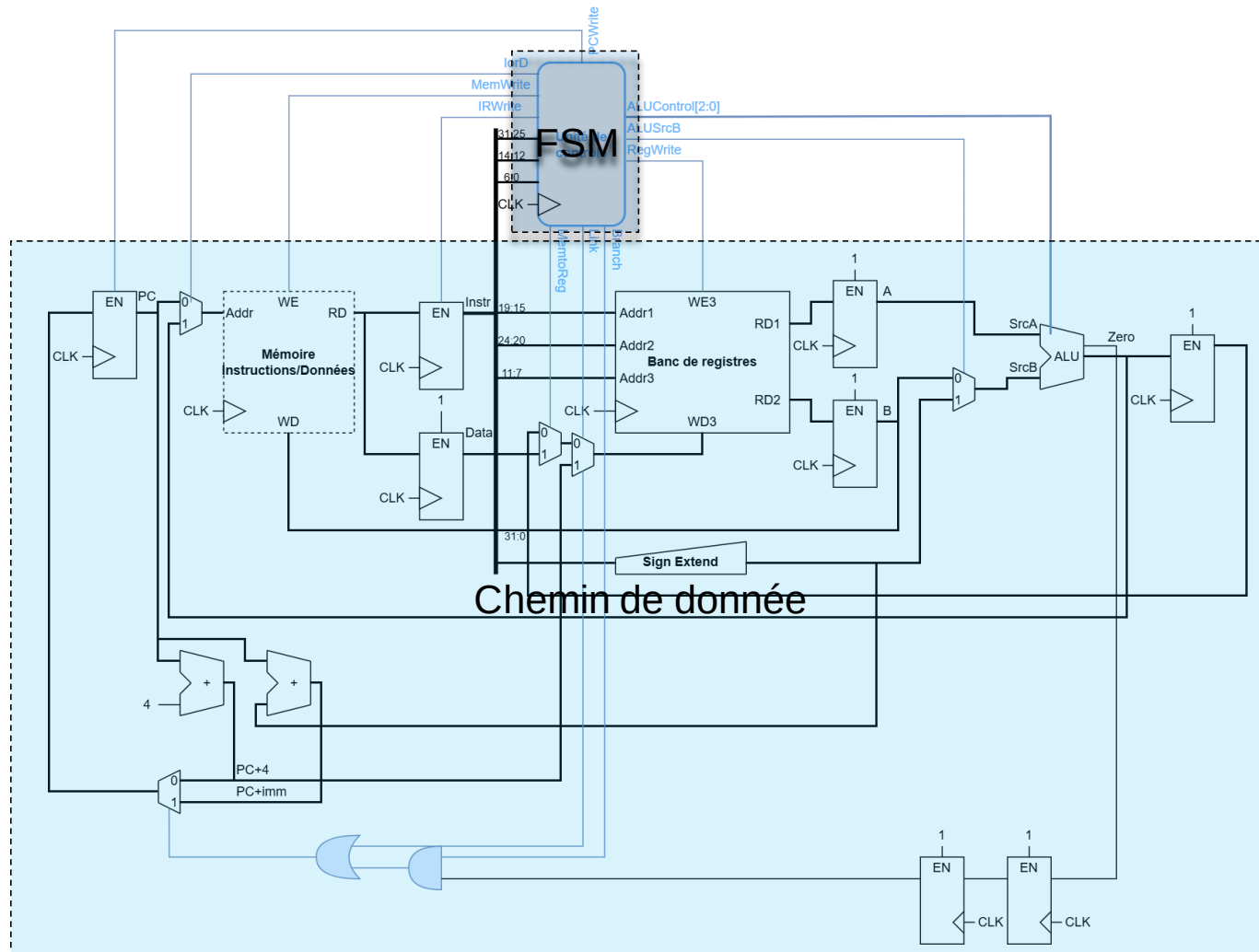
Exemple



Exemple d'une FSMD pour un processeur RISC-V multi-cycles (avec mémoire, PSN10)

Méthode combinée (ou FSMD)

Exemple

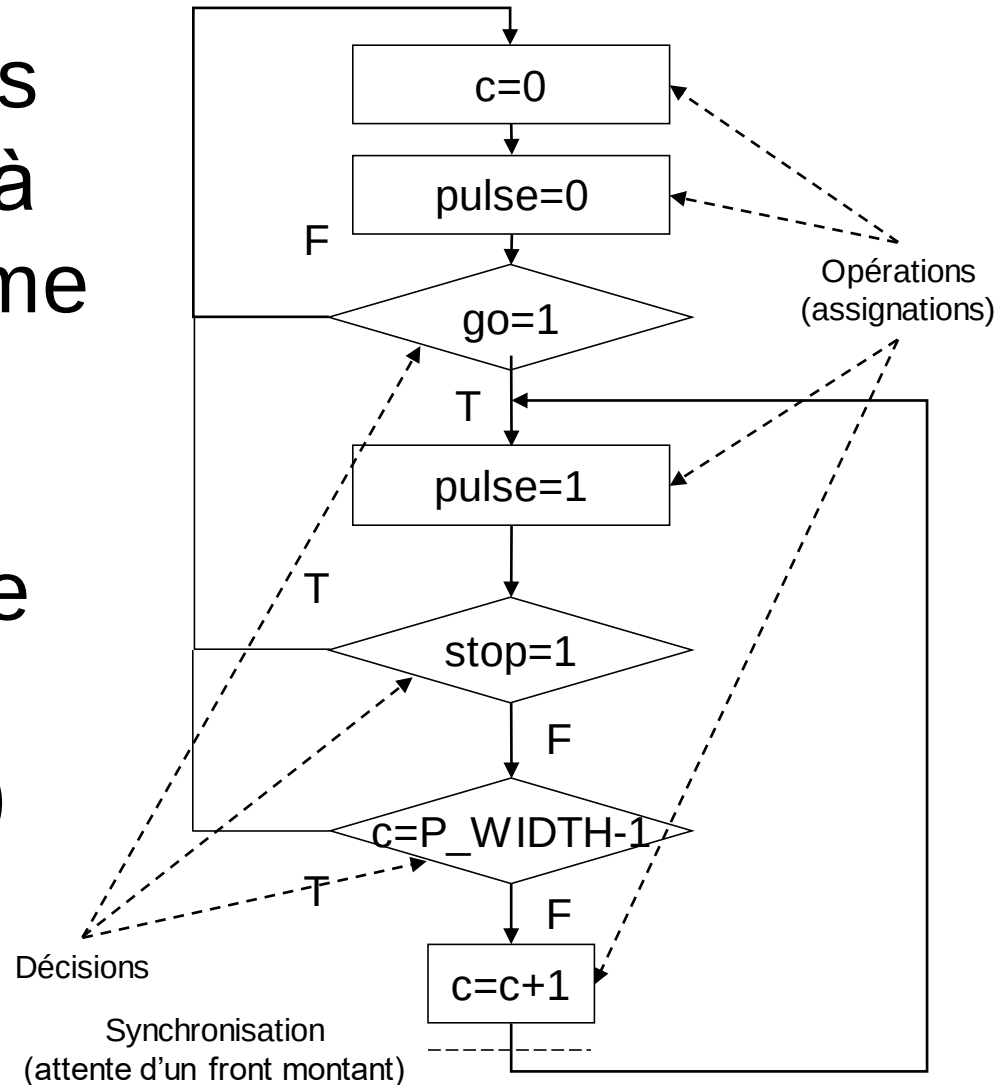


Exemple d'une FSMD pour un processeur RISC-V multi-cycles (avec mémoire, PSN10)

Conception RTL

Générateur d'impulsion

- Le cahier des charges exprime l'algorithme à implanter sous la forme d'un pseudo-code ou algorithme, aussi appelé organigramme de programmation (*flowchart* en anglais)



Conception RTL

Générateur d'impulsion (suite) – de l'algo. à l'ASMD

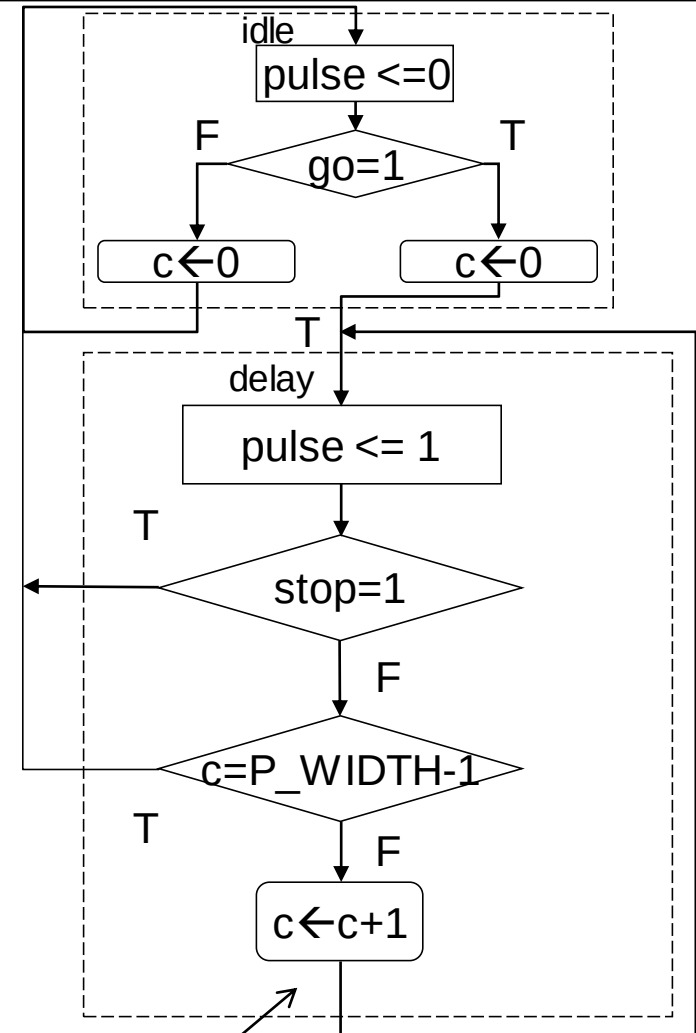
- Chaque opération de l'algorithme doit être affectée à un état (dans lequel plusieurs opérations peuvent être effectuées en parallèle)
- Le passage d'un état à un autre est contrôlée par une FSM
 - ➡ Spécifier sur l'algorithme les différents états de la FSM.
- L'algorithme devient alors l'ASMD :
Algorithm State Machine with Data path

Conception RTL

Générateur d'impulsion (suite) – de l'algo. à l'ASMD

Dans un même état (blocs en pointillés) les opérations ont lieu en parallèle (nécessite suffisamment de ressources matérielles disponibles)

Les affectations sont soit immédiates $\leftarrow$, soit différés $\leftarrow$



Sortie de Mealy (conditionnelles)

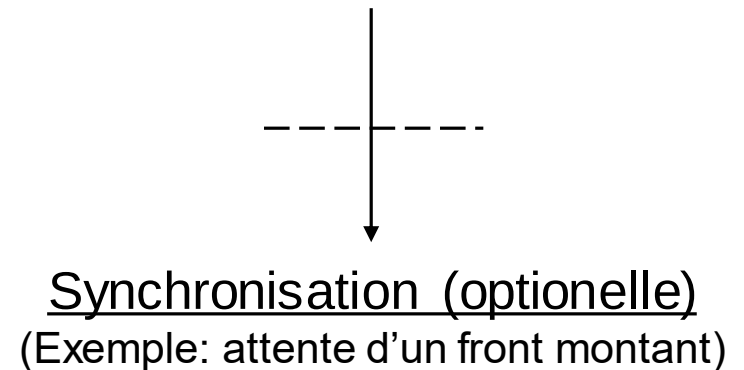
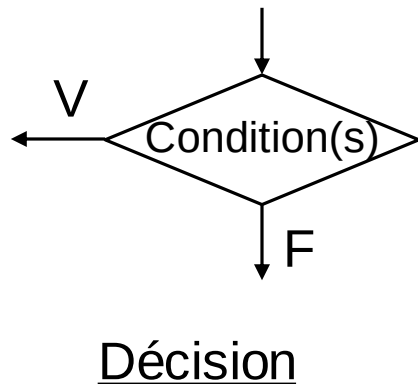
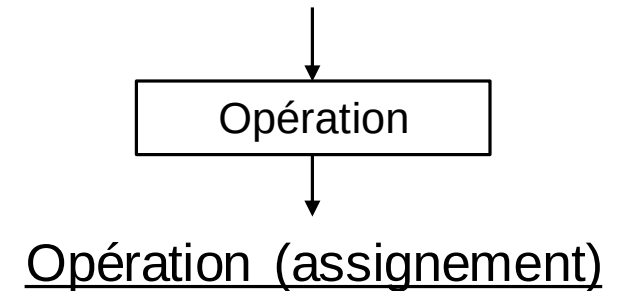
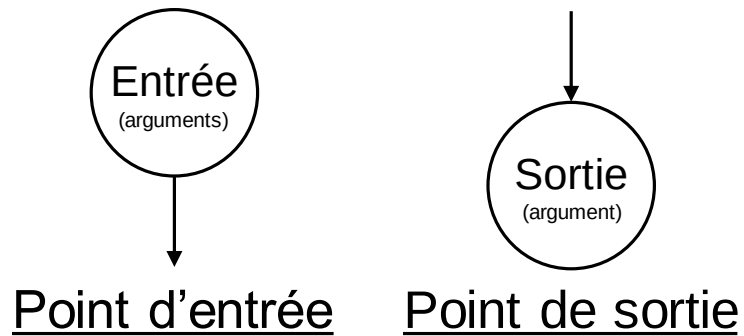
Méthode combinée (ou FSMD)

Etapes de la méthode

1. Exprimer le cahier des charges sous la forme d'un algorithme
2. Pré-traitement (optionnelle): Rajouter une structure de contrôle sur l'algorithme.
3. Transformer l'algorithme en un ASMD (ASM étendue):
 - a) Entourer en pointillé les blocs correspondant à des états
 - b) Transformer les affectations (immédiates) en des affectations immédiates ou différées
 - c) Modifier éventuellement les conditions

Méthode combinée (ou FSMD)

Algorithme



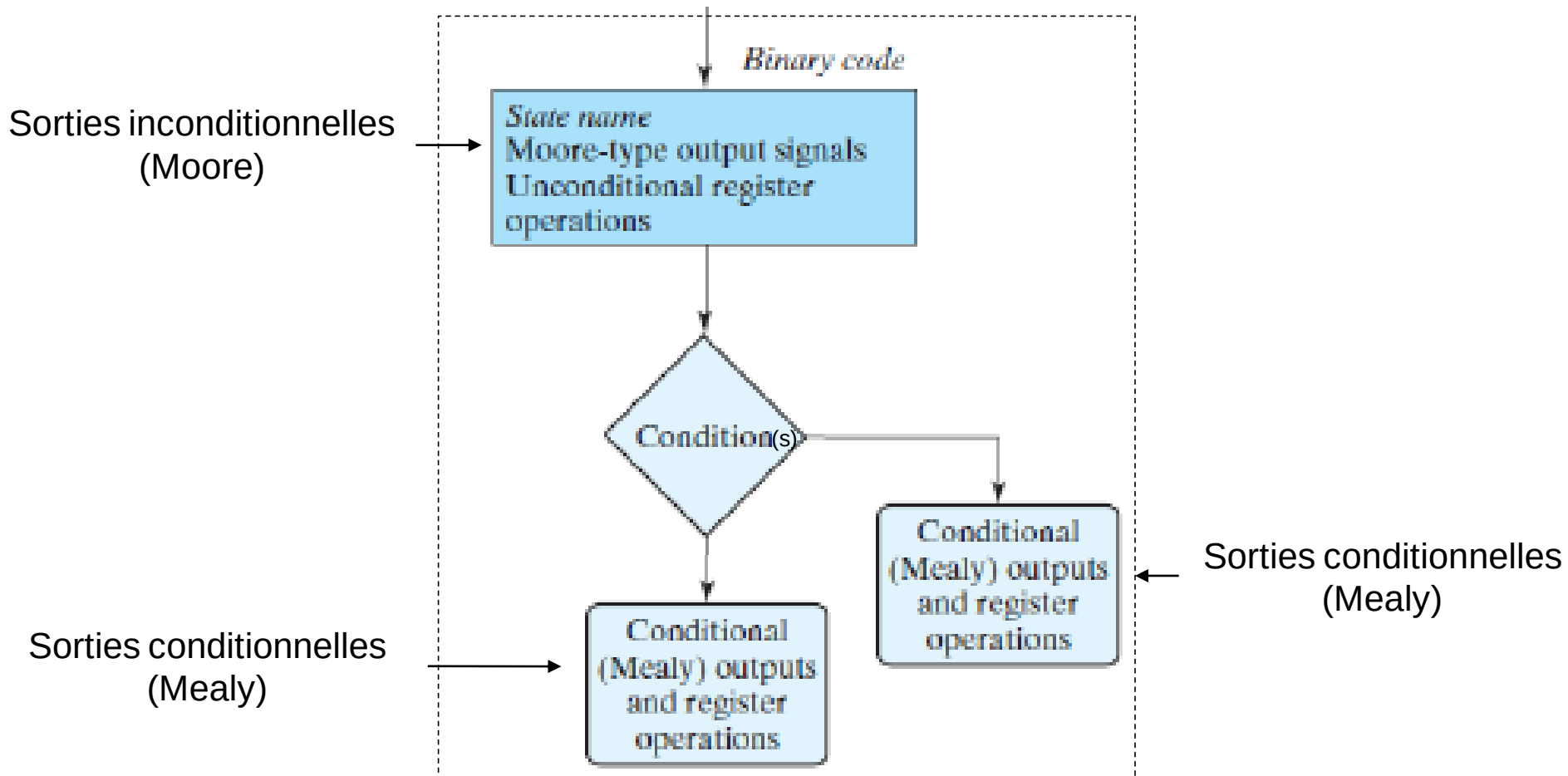
Méthode combinée (ou FSMD)

Pré-traitement

- Dans certains cas, l'algorithme contient que des points d'entrée/sortie et ne contient pas d'informations sur la manière d'interagir avec le circuit final.
- Il faut donc ajouter une structure de contrôle (avec des notions de synchronisation)
- Cela représente une vue haut-niveau de l'**interface** (entrées/sorties) du circuit.

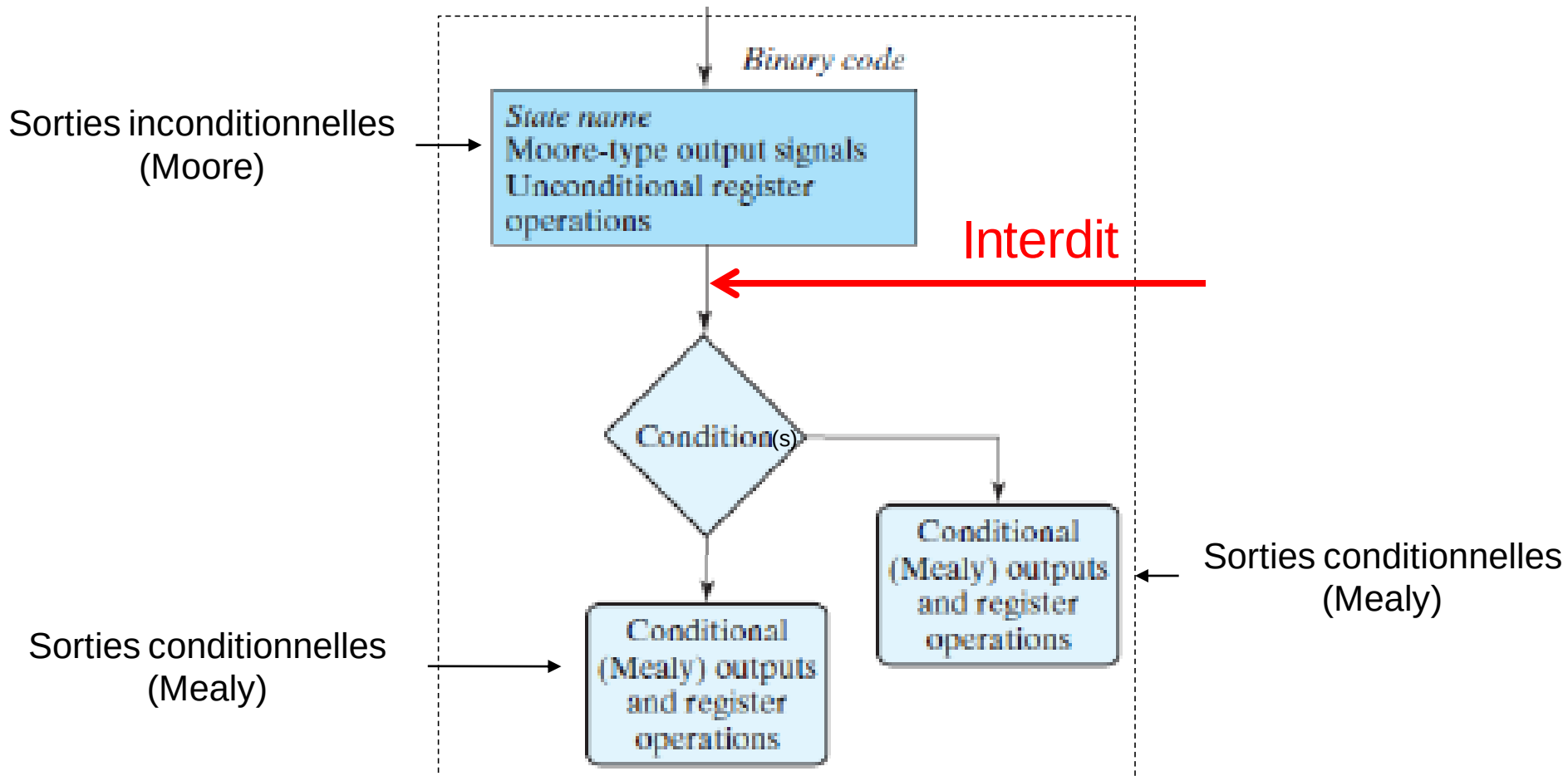
Conception RTL

a) déterminer les états de l'ASMD



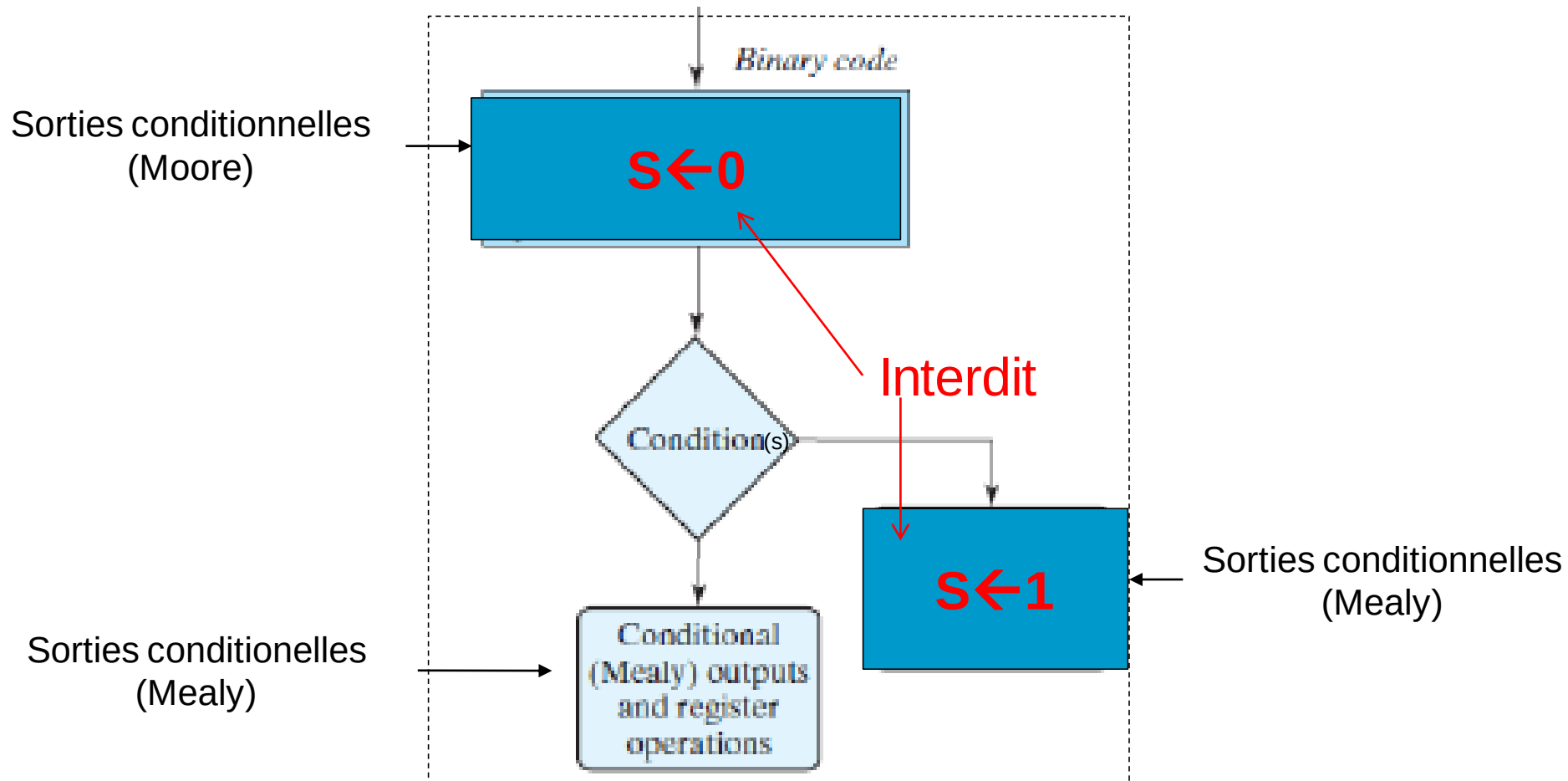
Conception RTL

a) déterminer les états de l'ASMD



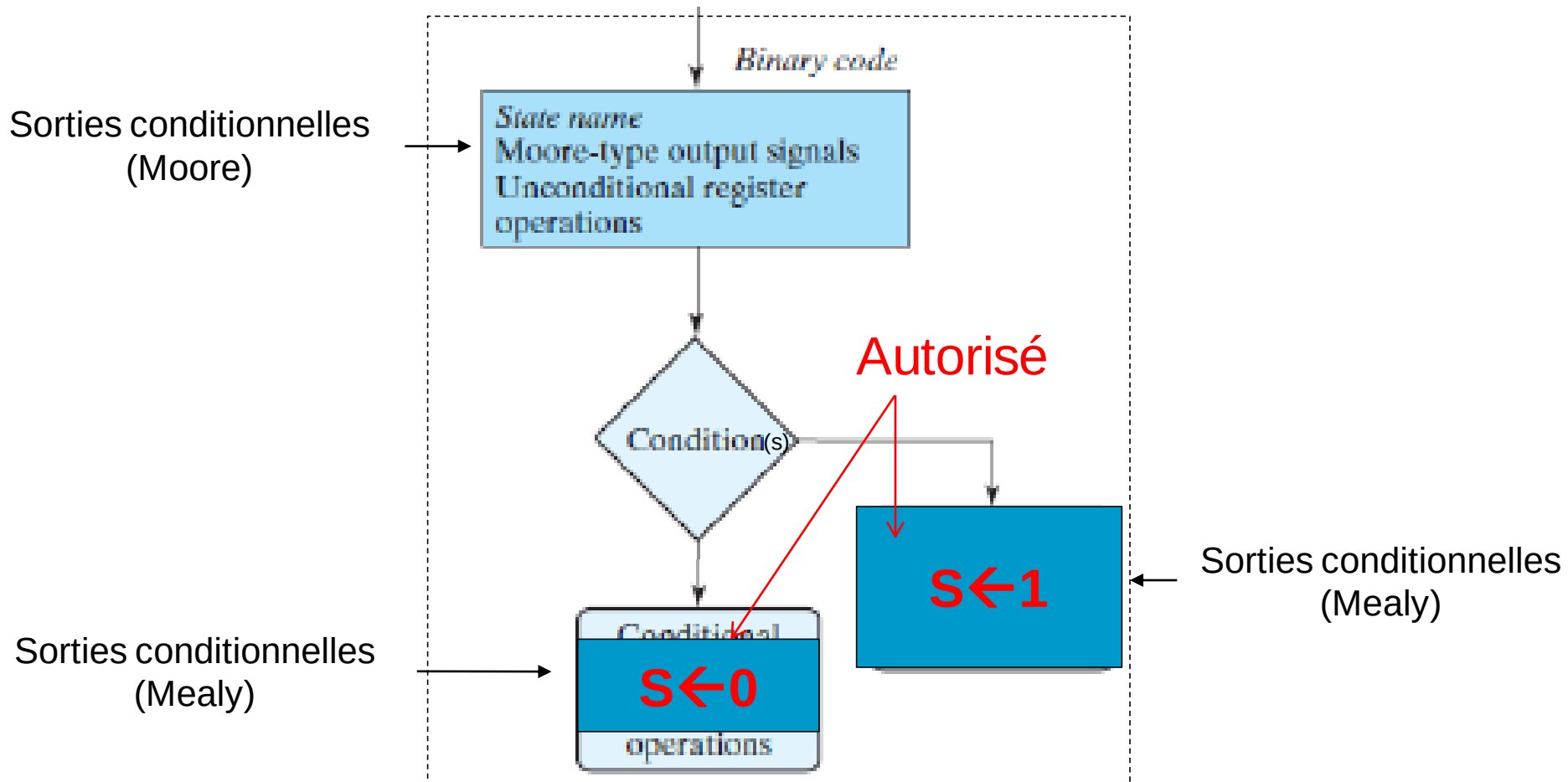
Conception RTL

a) déterminer les états de l'ASMD



Conception RTL

a) déterminer les états de l'ASMD



Conception RTL

a) déterminer les états de l'ASMD

- ASMD et l'algorithme ont des fonctionnements différents : parallèle vs. séquentiel.
- Le choix des états de l'ASMD influence les performances finales de l'implémentation : nombre de cycles de la FSM et nombre de ressources dans le *data path* (chemin de données)
- Attention à ne pas regrouper dans un même état, deux assignations en lien.
- Optimiser l'ASMD n'est pas une tâche triviale!

Conception RTL

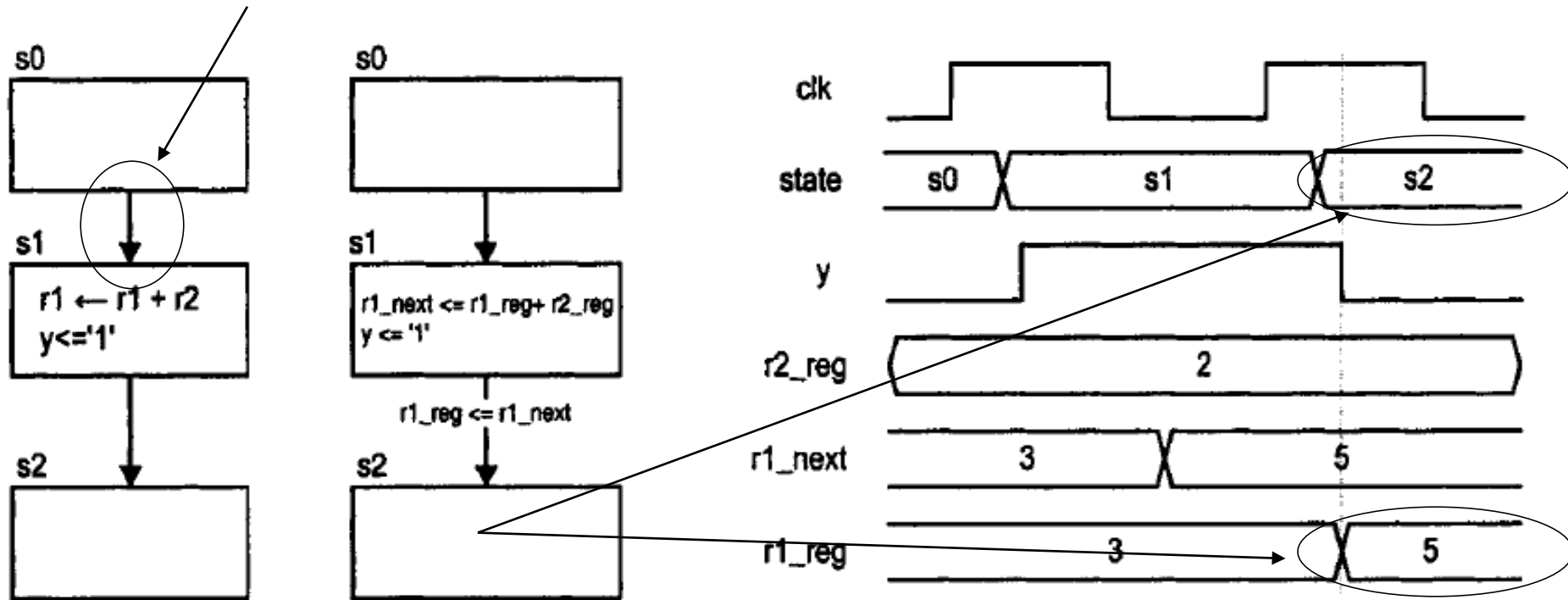
b) déterminer les affectations immédiates et différées

- Pour déterminer si une affectation est immédiate ou différée :
 - Les affectations immédiates concernent les affectations réalisées dans la partie contrôle (FSM), donc des valeurs fixes (souvent ce sont de simples bits)
 - Les affectations différées (micro-opérations) concernent les affectations réalisées dans la partie opérative (*data path*), donc elles concernent les signaux sur lesquels on a des opérations arithmétiques ou logiques (souvent ce sont des vecteurs)

Conception RTL

b) déterminer les affectations immédiates et différées

Transition synchrone (ici front montant de clk)

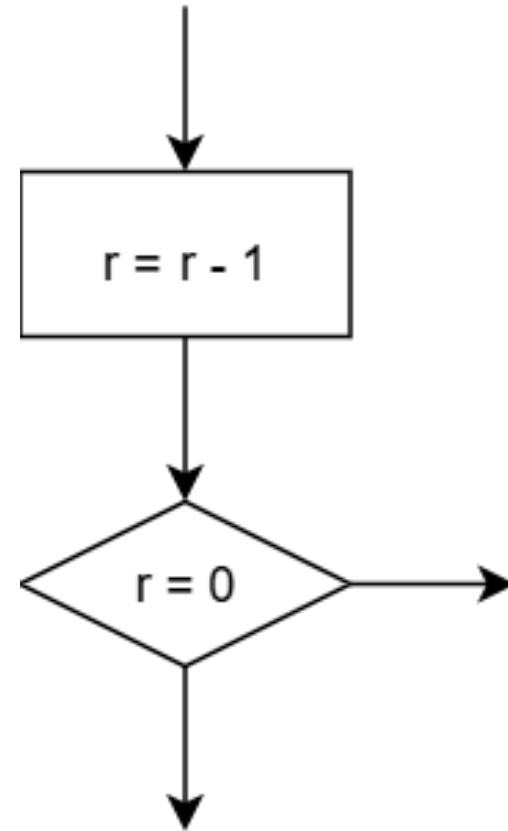


Attention! Les affectations $\leftarrow$ ont lieu **en sortant de l'état**
Alors que les affectations $\leq$ ont lieu **immédiatement**

Conception RTL

c) Conditions dépendant d'une opération précédente

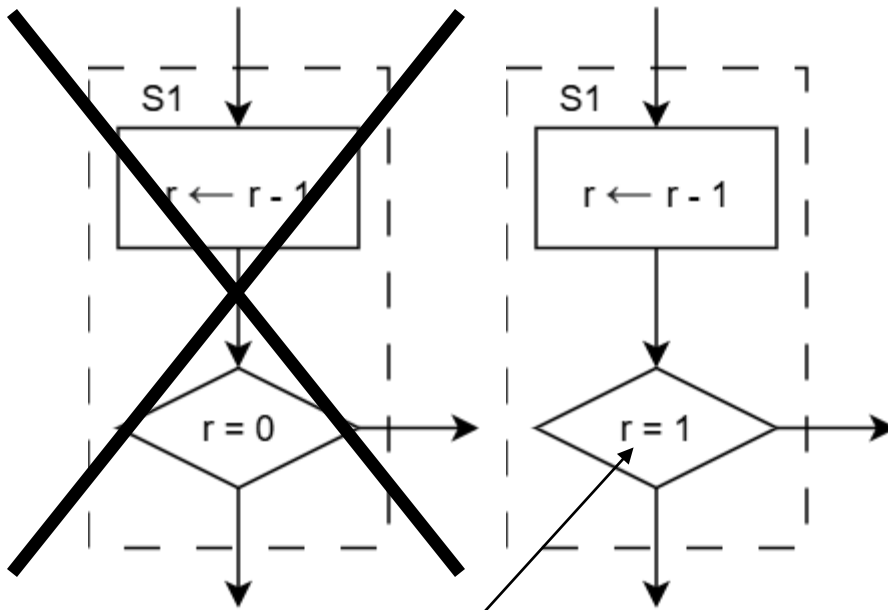
Quelle différence entre algorithme et ASMD quand la variable est utilisée dans une condition ?



Conception RTL

c) Conditions dépendant d'une opération précédente

➤ Problème de la synchronisation des conditions

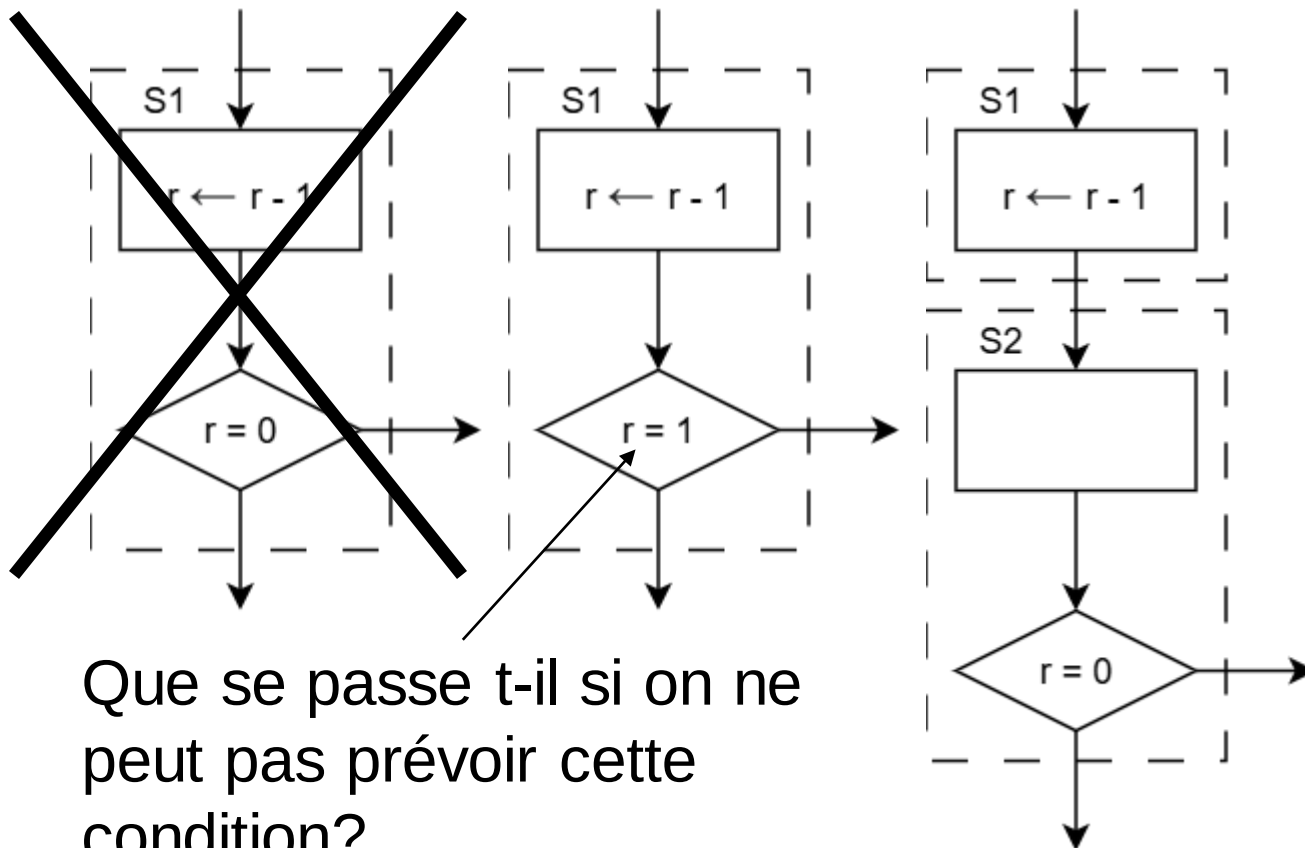


Que se passe t-il si on ne peut pas prévoir cette condition?

Conception RTL

c) Conditions dépendant d'une opération précédente

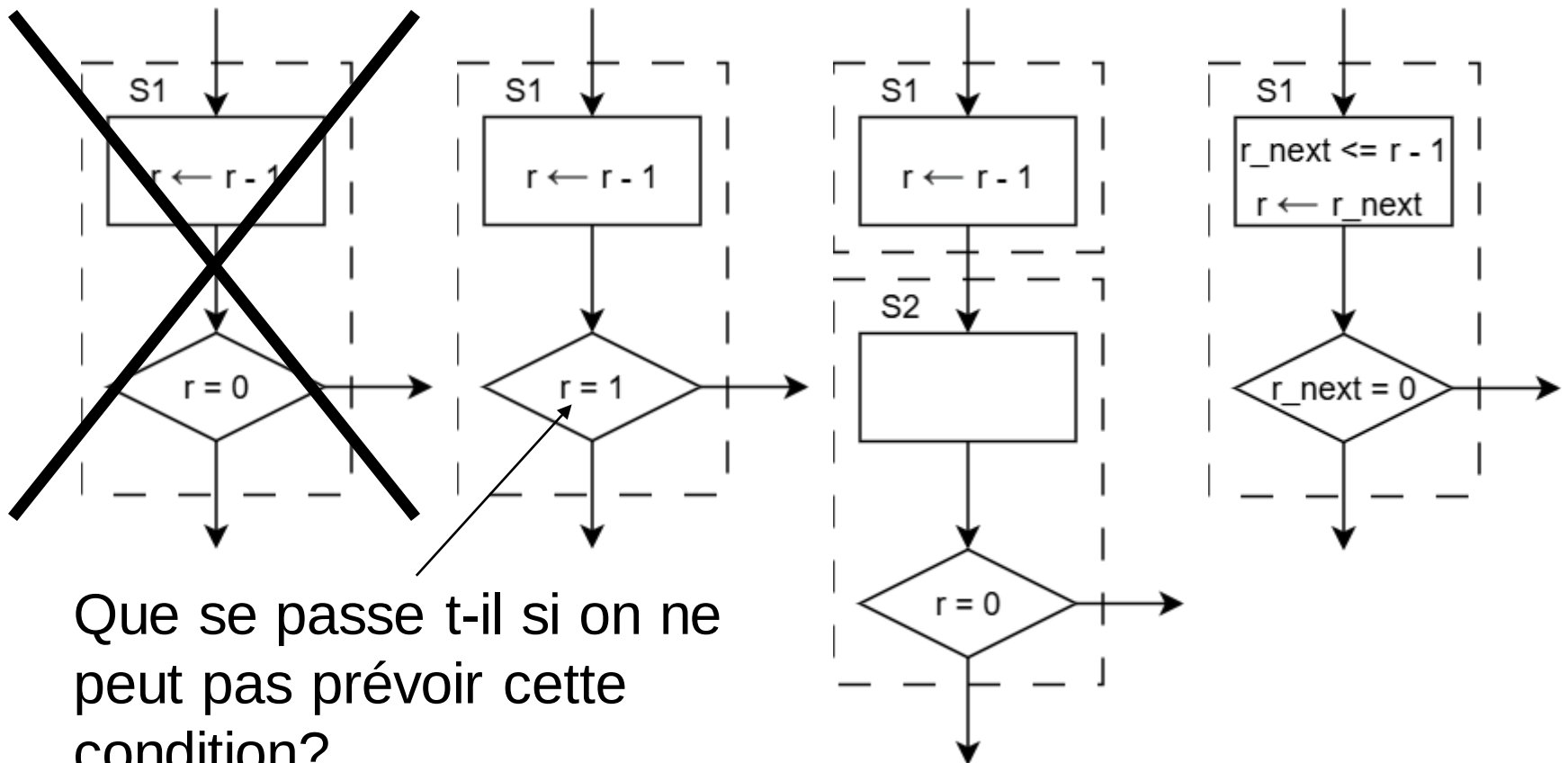
➤ Problème de la synchronisation des conditions



Conception RTL

c) Conditions dépendant d'une opération précédente

➤ Problème de la synchronisation des conditions



Conception RTL

VHDL

```
architecture fsmd_arch of pulse_5clk is  
constant P_WIDTH : natural := 5;  
type fsmd_state_type is (idle , delay);  
signal state_reg , state_next : fsmd_state_type;  
signal c_reg, c_next: unsigned(2 downto 0); --valeur max 4
```

- *state and data registers*

```
process(clk, reset)  
begin  
    if (reset='1') then  
        state_reg <= idle;  
        c_reg <= (others=>'0');  
    elsif (clk'event and clk='1') then  
        state_reg <= state_next;  
        c_reg <= c_next;  
    end if;  
end process;
```

Conception RTL

VHDL next_state logic & datapath

```
process (state_reg ,go, stop, c_reg)
begin
    pulse <= '0'; c_next <= c_reg;
    case state_reg is
        when idle =>
            if go='1' then
                state_next <= delay;
                c_next <= (others => '0');
            else
                state_next <= idle;
                c_next <= (others => '0');
            end if;
        when delay => pulse <= '1';
            if stop='1' then
                state_next <= idle;
            else
                if (c_reg=P_WIDTH-1) then
                    state_next <= idle;
                else
                    state_next <= delay;
                    c_next <= c_reg + 1;
                end if;
            end if;
        end case;
    end process;
end fsmd_arch ;
```

Les FSMD sont plus faciles
à comprendre et donc à modifier
(que les solutions régulières
ou aléatoires)

Sorties de Mealy :
les sorties (ici c_next)
dépendent
des entrées

Conception RTL

Exemple : multiplieur séquentiel

- On se propose de construire une architecture qui exécute l'algorithme classique de la **multiplication binaire**.

$$f(a, b) = \begin{cases} a + f(a, b - 1) & \text{si } b > 0 \\ 0 & \text{sinon} \end{cases}$$

Conception RTL

Exemple : multiplieur séquentiel

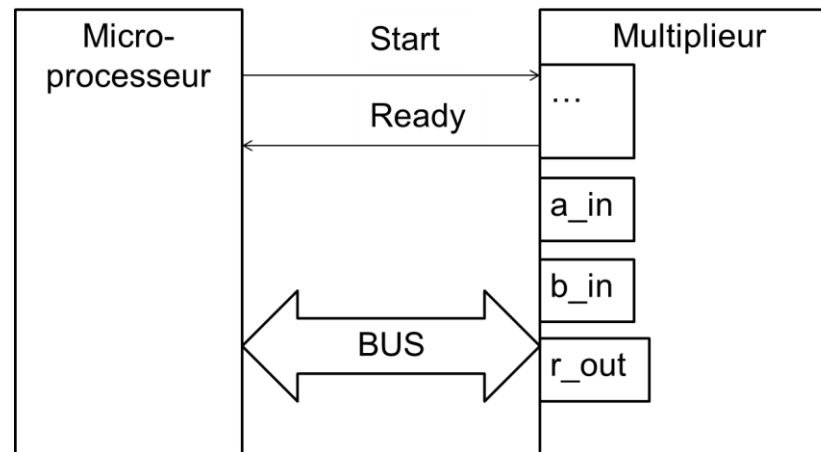
Cahier des charges :

Étape initiale : Les deux opérandes sont présentés sur deux ports d'entrée *a_in* et *b_in*, le départ de l'exécution de la multiplication est signalé par *start*='1'.

Le circuit répond alors *ready*='0' (« occupé ») pendant tout le temps de calcul

Étape 2 : Exécution de l'algorithme

Étape finale : Lorsque *ready*='1', le résultat est présenté sur le port *r_out*. Le circuit retourne à l'étape initiale.



Conception RTL

Exemple : multiplieur séquentiel - pseudo-code&alg.

entier multiplication(entier a_bin, entier b_in)

 a = a_in

 n = b_in

 r = 0

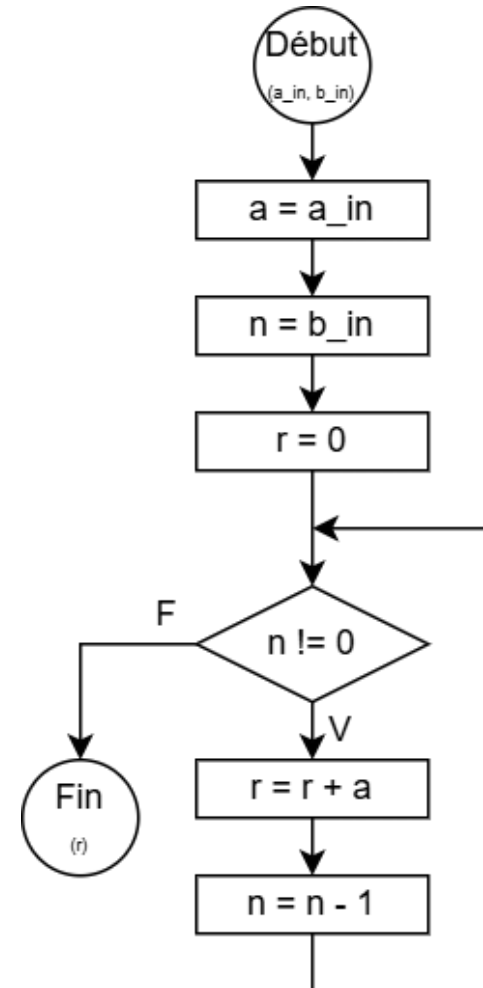
 tant que (n != 0)

 r = r + a

 n = n - 1

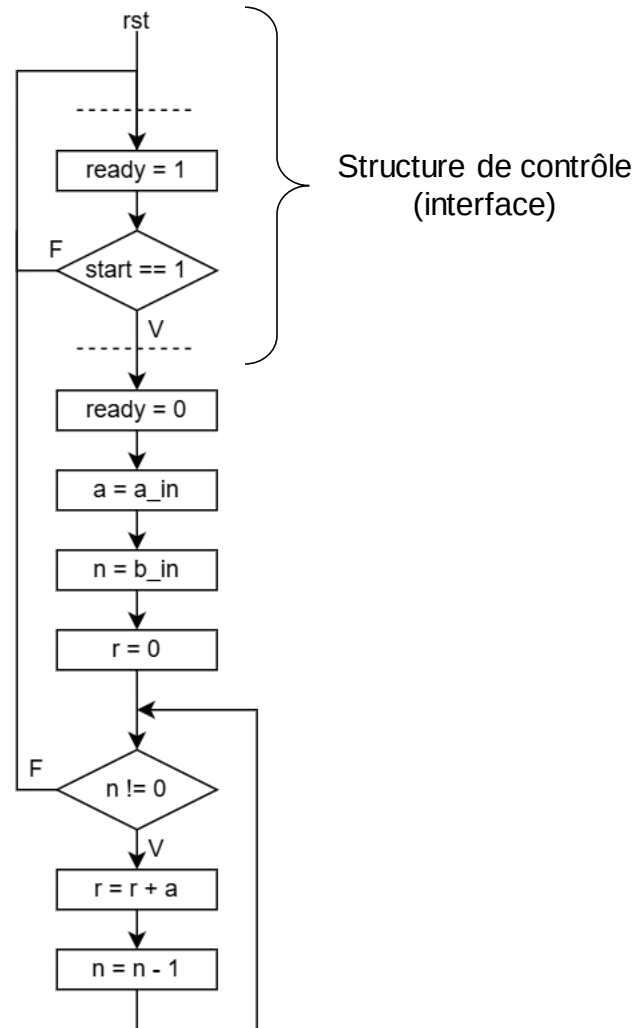
 retourner r

Pseudo-code de la multiplication
séquentielle



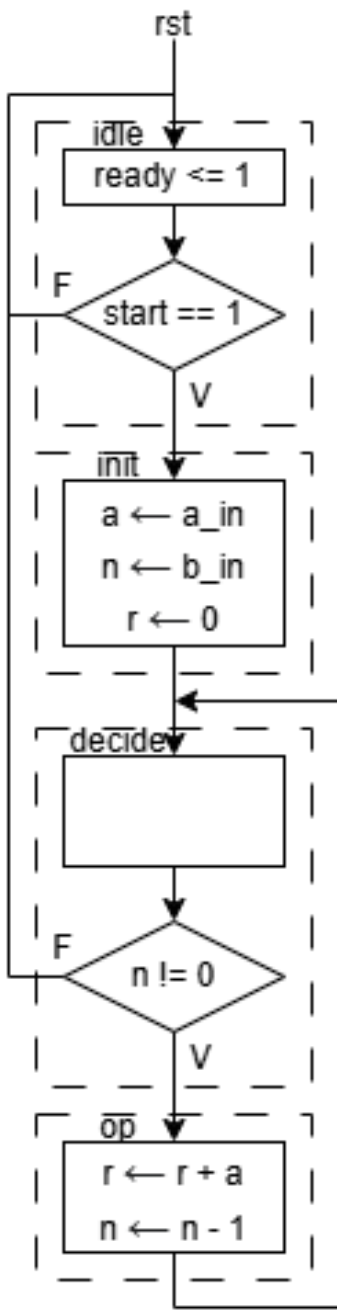
Conception RTL

Exemple : multiplieur séquentiel - Interface.



- Entrées:
 - a_in, b_in : 8-bit unsigned
 - clk : signal d'horloge
 - reset : signal de reset
 - start : commande
- Sorties:
 - r_out <= r : 16-bit unsigned
 - ready : status
- Valeurs par défaut
 - ready <= 0
 - a ← a
 - n ← n
 - r ← r

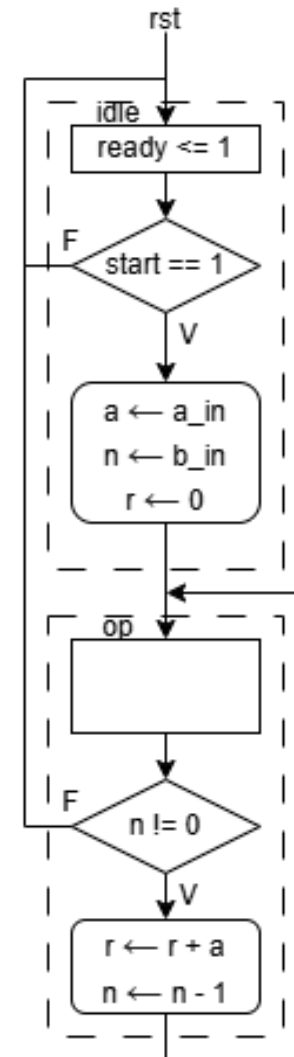
Sorties de Moore :
Une unique affectation
des sorties par état



Conception RTL

Exemple du multiplieur séquentiel – Machine de Mealy

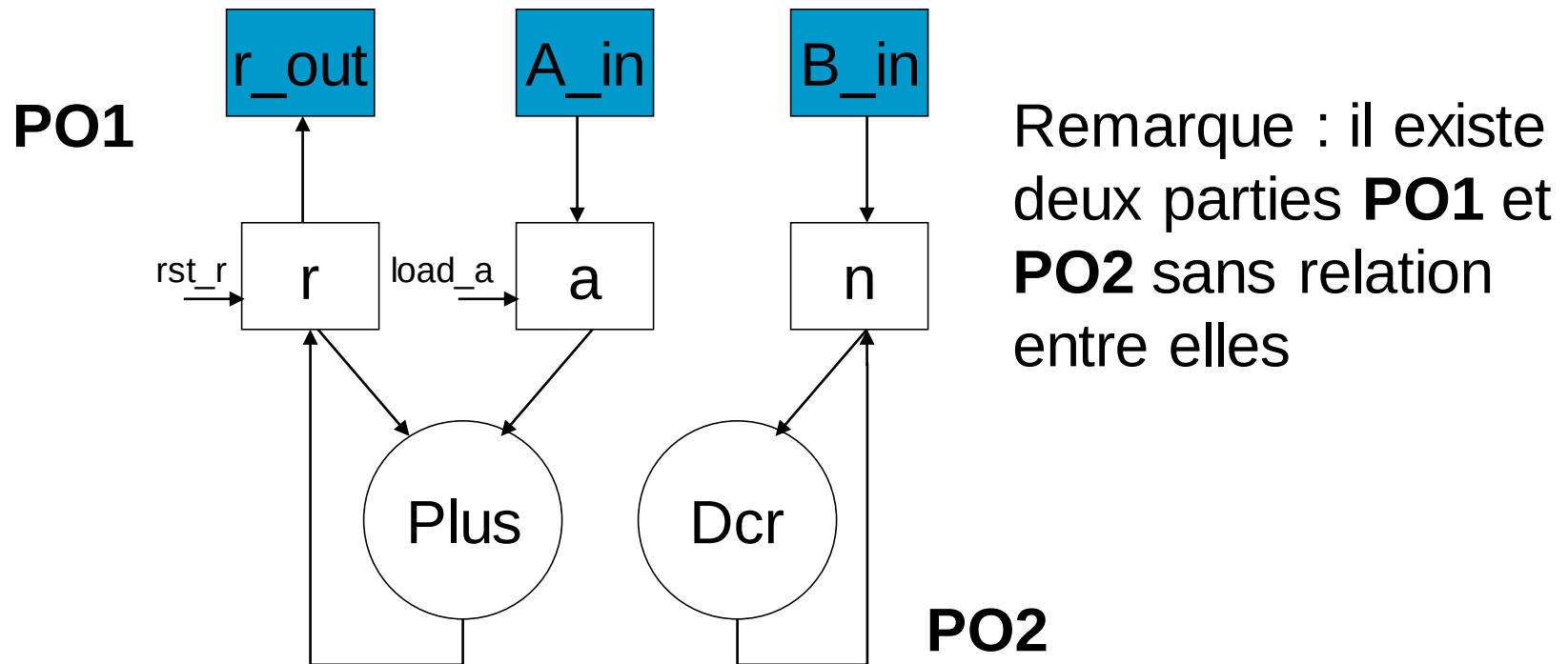
- Utilisation de sorties de Mealy pour l'implémentation de la partie calcul (*data path*)
- Possibilité d'affecter différentes valeurs à un même registre suivant les différentes conditions au sein d'un même état.
- Elimination de l'état *init* (intégré dans *idle*) et de l'état *decide* (intégré dans *op*)



Conception RTL

Exemple du multiplieur séquentiel

Description du chemin de données (*data path*)



MUX pour éviter les conflits non représentés

Conception RTL

Exemple : multiplieur séquentiel

- Optimisations algorithmiques possibles:
 - Vérification de la valeur de a_in (et b_in) en début de l'algorithme, pour éviter de lancer le calcul si une des deux valeurs est nulle.
 - Comparaison de la valeur de a_in et b_in , pour limiter le nombre d'itérations ($a=b_in$ et $n=a_in$).
 - ...
- Optimisations de l'ASMD possibles:
 - Partage des opérateurs de calcul, en séparant $r \leftarrow r + a$ et $n \leftarrow n - 1$ dans deux états différents.
 - ...

Conception RTL

Exemple du multiplieur séquentiel

Voir le code VHDL sur le document annexe intitulé :

Conception RTL

Détail de la conception d'un multiplieur séquentiel

Conception RTL

Exercice : Suite de Fibonacci

- Construire l'ASMD permettant d'implanter la suite de Fibonacci définie à partir des équations de récurrence ci-dessous :

$$fib(i) = \begin{cases} 0 & \text{if } i = 0 \\ 1 & \text{if } i = 1 \\ fib(i - 1) + fib(i - 2) & \text{if } i > 1 \end{cases}$$

(la solution est disponible sur Chamilo)

Exercices

- **Faire le QCM associé à ce chapitre sur Chamilo**